

IMPROVING DEEP REINFORCEMENT LEARNING METHODS FOR PROTEIN STRUCTURE PREDICTION IN THE 2D HYDROPHOBIC-POLAR MODEL

Yash Shah & Tali Motzkin

Department of Computer Science

University of Cambridge

{ys562, tm861}@cam.ac.uk

ABSTRACT

We aim to extend existing work regarding the 2D Hydrophobic-Polar protein folding problem in a few ways. Firstly, we explore improvements and alternatives to DQN. Secondly, we aim to create a general reinforcement learning agent that can fold multiple sequences, without being required to be retrained from scratch. We also investigate transitioning away from LSTM-based architectures to better optimize longer sequences. Lastly, we demonstrate the importance of capturing long-range dependencies by the model architecture for generating optimal solutions to this problem. **Yash**

1 INTRODUCTION

1.1 OVERVIEW OF THE PROTEIN FOLDING PROBLEM

8276 Our project aims to tackle the HP protein folding problem using reinforcement learning. Before we delve into the specifics, it's crucial to understand the importance of protein folding. Proteins, made up of amino acids, fold in a cell into specific three-dimensional structures that determine their functions. This precise folding is critical for the protein to function as intended. For instance, many proteins serve as enzymes that catalyze essential biochemical reactions. Incorrect folding can render these enzymes inactive, disrupting normal biological processes. Furthermore, misfolding of proteins is linked to aging and human diseases (Daggett (2006)). A deep understanding of protein folding can aid in preventing these diseases, developing effective treatments, and help us better understand nature (Dobson (2003)). **Tali**

Protein folding is driven by the need to achieve the lowest free energy, resulting in the formation of their native structure (Anfinsen (1973)). As a thermodynamic process, proteins naturally progress toward the most stable and energetically favorable structure or conformation (Honig and Yang (1995)). In fact, a small protein can exhibit approximately 10^{30} main-chain conformations. The immense scope of these possibilities implies that theoretically, exploring all these conformations could take about 10^{11} years (Dinner et al. (2000)). However, in nature, proteins fold within seconds, making protein folding a fascinating search optimization problem. **Tali**

1.2 THE HP PROTEIN FOLDING PROBLEM

To simplify the problem, a 2D HP model was considered, where proteins are represented by sequences of hydrophobic (H) and polar (P) amino acid units (Yang et al. (2023)). Therefore, in the HP protein folding problem, the goal is to identify a folding pattern that maximizes the number of H-H contacts, as these correspond to the lowest energy state and thus the most stable configuration of the protein. **Tali**

Hydrophobic substances tend to aggregate to reduce their exposure to water, due to enthalpic and entropic effects (Atwood (2017)). This is known as the Hydrophobic Effect. As a result, hydrophobic regions collapse into a compact core, minimizing the system’s free energy. The interactions between spatially neighboring amino acids can be broken down into local interactions among monomers near each other in the protein chain, and non-local interactions between monomers with wide separations along the sequence (Dill et al. (1995)). By taking the view that non-local interactions dominate, the Hydrophobic Effect pulls together hydrophobic regions (H), which in turn pulls the protein’s backbone along, resulting in the protein folding. It is important to note that such hydrophobic interactions (characterized by H-H contacts) are not the only monomer interactions governing the folded structure of the protein, but the 2D HP model serves as a computationally convenient simplification of the problem (Hoque et al. (2009)). Yash

1.3 CODEBASE

Our code can be found in <https://github.com/Paroxysmisch/hp-model> Yash for evaluating different model architectures and <https://github.com/TaliMotzkin/hp-model/tree/dev-tali> Tali for testing various agents and experimenting with generalized environment.

2 RELATED WORK AND ITS LIMITATIONS

Since the protein folding problem in the HP model is NP-complete (Berger and Leighton (1998)), a range of approximation and heuristic methods have been explored to address it (Czibula et al. (2011)). These methods include Evolutionary and Genetic algorithms, Growth algorithms, Reinforcement Learning (RL) etc. Several works have found that RL techniques can effectively tackle the protein folding problem using the 2D HP model (Doğan and Ölmez (2015)). One of the first attempts was by Czibula et al. (2011), using tabular Q-learning on a short HP sequence. Another work by Doğan and Ölmez (2015) changed the state-action representation to be more search efficient and combined it with the Ant-Q learning algorithm (Gambardella and Dorigo (1995)), solving for sequences up to a length of 48. Additionally, Wu et al. (2019) applied similar methods but experimented with variations in the reward function. Tali

However, the scalability issues of tabular Q-learning likely prompted researchers to explore other paradigms such as deep learning based RL approaches. The first article to apply DRL to the protein folding problem was by Li et al. (2018). Inspired by AlphaGo Zero, their approach utilized a two-head deep convolutional neural network (HPNet) coupled with regularized Upper Confidence Bounds for Trees (R-UCT). In their strategy, the policy head of HPNet is used to expand the unexplored leaf nodes rather than performing traditional rollouts. Yu et al. (2020) applied various DRL methods to the HP model, utilizing techniques such as DQN, Monte Carlo methods, a Prioritized Replay Buffer, and a generalized pre-trained model similar to AlphaGo Zero, with architectures ranging from simple MLPs to more complex CNNs. They noted that the pre-trained model outperformed the other approaches, though detailed comparative analysis was not clearly stated. Tali

The recent paper by Yang et al. (2023), which serves as the primary basis for our work, introduced a novel approach by integrating a Deep Q-Network (DQN) with an LSTM model, achieving the best known energies for protein sequences up to a length of 50. Table 2 in their publication provides a comparative analysis of their results against those from previously mentioned studies. Notably, none of the studies have conducted experiments on protein sequences of length 100. Tali

We identified a number of limitations of past work Yash:

1. DQN is a value-based approach, but policy-based approaches may allow for smoother exploration of the actions space through direct policy optimization, along with stronger convergence guarantees (Sutton and Barto (1998)).

2. LSTM (Yu et al. (2019)) has been the dominant model architecture used for this problem, but is weak regarding long-range dependency modelling (Khandelwal et al. (2018)).
3. Prior work utilizes Deep Q-Networks (DQN), which are known to suffer from the overoptimism due to estimation errors (Van Hasselt et al. (2016a)). These overestimations of action values may significantly harm the system’s performance.
4. Previous RL approaches utilized a queue-based replay buffer with random sampling to enhance sample efficiency. However, this approach results in transitions being replayed at the same frequency as they were originally experienced. Prioritized experience replay (Schaul et al. (2015a)) could potentially speed up training by replaying more important transitions with increased frequency.
5. Existing work utilizing reinforcement learning for the HP protein folding problem retrains a new model for each HP sequence, potentially neglecting gains from cross-sequence generalization.

3 OUR CONTRIBUTION

We aim to mitigate each one of the limitations identified above as follows Yash:

1. Assessing the viability of PPO as a replacement for DQN (and DDQN).
2. Analyzing the performance of newer model architectures (Mamba, Transformer, and GAT) that can better capture long-range dependencies, empirically finding evidence for the need for modelling these dependencies, and viewing the problem in a more principled manner via a Geometric Deep Learning (Bronstein et al. (2017)) framework.
3. Evaluating the use of Double Deep Q-Networks (DDQN) in addressing the overoptimism problem.
4. Integrating prioritized experience replay.
5. Testing generalization capabilities across HP sequences via curriculum learning.

More details about each of these strategies is provided in the subsections below. Yash

3.1 EXPLORING DIFFERENT AGENTS—POLICY GRADIENT METHODS

We considered policy gradient methods, which introduce another level of sophistication, potential improvement and offer stronger convergence guarantees (Sutton and Barto (1998)). By learning a probability distribution over actions, policy gradient methods inherently support stochastic policies (such as in our case) and can smoothly transition to deterministic policies asymptotically during training (Sutton and Barto (1998)). Tali

In the debate over which policy gradient method might be most effective for our case, REINFORCE initially appears promising due to its simplicity and direct applicability to episodic reward structures of the HP model. However, REINFORCE suffers from high variance in its estimates (Sutton and Barto (1998)), which can slow down convergence and lead to less stable training. Tali

On the other hand, Proximal Policy Optimization (PPO) offers a more balanced approach. It retains the benefits of policy gradient methods, reduced variance through stochastic gradient ascent on mini-batch updates, and through a clipped surrogate objective function. This reduces the risk of having policy updates that are too large, hence improving training stability (Schulman et al. (2017)). Also, performing as the “winning” algorithm of many problems (Liu et al. (2019)), we choose to implement it in our model. Tali

3.2 EXPLORING ALTERNATE MODEL ARCHITECTURES

Prior work (Yang et al. (2023)) has focused on the use of Recurrent Neural Networks (RNNs), specifically Long Short-Term Memory (LSTM) networks (Yu et al. (2019)) for modelling of the state-action value function. While this has proven to be effective on small HP sequences ($N \leq 50$), tackling larger problems likely requires a shift away from recurrent architectures, which suffer from two major issues. (Yash)

Firstly, the fixed hidden state size of LSTMs serves as a memory bottleneck (Vaswani et al. (2017)), hindering the ability of the model to capture long-range dependencies and global context. Secondly, it has been shown that LSTMs are capable of sharply distinguishing nearby context, but model the distant past (> 50 tokens) merely as a rough semantic field (Khandelwal et al. (2018)). While this is perhaps acceptable for many language modelling tasks, it is certainly of paramount concern here, since the model must retain clear memory of the complete laid-out sequence to optimize the placement at each timestep. (Yash)

We believe that moving away from RNNs can improve HP protein folding performance, particularly on longer sequences. (Yash)

3.2.1 INVESTIGATING MAMBA SELECTIVE STATE SPACE MODEL

Attention has proven to be effective in routing information within its context window, but its computational cost scales quadratically with the sequence length. More efficient variants of the attention mechanism have failed to perform well empirically across domains. In contrast, Structured State Space Models (SSMs) (Gu et al. (2021)), which can be viewed as a hybrid of Recurrent and Convolutional Neural Networks, have linear, or near-linear scaling in the sequence length, and handle long-range dependencies well. However, SSMs have had trouble modelling discrete, information-dense data, which leads to the Mamba *Selective* State Space Model, which overcomes this limitation by effectively selecting data in an input-dependent manner. (Yash)

Mamba (Gu and Dao (2023)) could help in efficiently handling long HP sequences, while maintaining the crucial long-range context required for optimal folding, and we thus investigate this architecture. (Yash)

3.2.2 A BRIEF OVERVIEW OF MAMBA

Mamba is derived from the class of Structured State Space Models (S4) (Gu and Dao (2023)). They can be viewed as a combination of recurrent and convolutional networks. While S4 models have a recurrent formulation, the result of applying them to a sequence can be viewed as a parallelizable convolutional operation, making them extremely fast. Importantly, these models have principled mechanisms for long-range dependency modelling (Gu et al. (2020)), performing well in benchmarks such as Long Range Arena (Tay et al. (2020)). S4 models, defined with parameters $(\Delta, \mathbf{A}, \mathbf{B}, \mathbf{C})$ can be formulated as follows:

$$\begin{aligned} h'(t) &= \mathbf{A}h(t) + \mathbf{B}x(t) & h_t &= \bar{\mathbf{A}}h_{t-1} + \bar{\mathbf{B}}x_t & \bar{\mathbf{K}} &= (C\bar{\mathbf{B}}, C\bar{\mathbf{A}}\bar{\mathbf{B}}, \dots, C\bar{\mathbf{A}}^k\bar{\mathbf{B}}, \dots) \\ y(t) &= \mathbf{C}h(t) & y_t &= \mathbf{C}h_t & y &= x \star \bar{\mathbf{K}} \end{aligned}$$

The first column refers to a continuous system, mapping a *continuous* sequence $x(t) \in \mathbb{R} \rightarrow y(t) \in \mathbb{R}$, through an implicit hidden latent space $h(t) \in \mathbb{R}^N$. Since we are interested in modelling *discrete* data, the continuous parameters $(\Delta, \mathbf{A}, \mathbf{B})$ are transformed to discrete parameters $(\bar{\mathbf{A}}, \bar{\mathbf{B}})$ through a discretization rule. The second column is the result after applying the discretization rule. By expanding and simplification (through the Linear Time Invariance (LIT) property due to the fact that $(\Delta, \mathbf{A}, \mathbf{B})$ and $(\bar{\mathbf{A}}, \bar{\mathbf{B}})$ remain constant for all time-steps) of the equations in the second column, we find that the transformation across the entire sequence x can be modelled as a convolution operation with kernel $\bar{\mathbf{K}}$, resulting in the output vector for all timesteps y , shown in the last column. S4 models are

structured due to constraints imposed on $\mathbf{A}$, such as it being a diagonal matrix to speed up computation. Yash

Although S4 can model recurrent processes with latent state across large contexts well (perfect for modelling Markov Decision Processes (MDPs) (Hafner et al. (2019))), they are not as effective at modelling discrete, information-dense data. By introducing the ability of the model to select data in an input-dependent manner, we create a model that can filter out irrelevant data, and keep relevant information indefinitely. However, this breaks the time- and input-invariance that makes the convolution operation possible in S4, responsible for S4’s speed. This is compensated for by replacing the convolution with a scan/prefix sum operation. Together, this results in the Mamba *Selective State Space Model*. Yash

3.2.3 INVESTIGATING THE USE OF TRANSFORMER

There has been an attempt at improving the LSTM’s ability to capture these long-range dependencies in the HP protein folding problem specifically, through the addition of the attention mechanism (Bahdanau et al. (2014)) in work by Espitia et al. (2024), to augment the underlying recurrent architecture. We believe that better performance can be achieved by removing reliance on RNNs. Yash

The Transformer model (Vaswani et al. (2017)) has been critical in advancing language modelling, and could play a role in improving the performance of the HP protein folding task too. We explore the result of solely using attention i.e. the Transformer. Yash

3.2.4 TRANSFORMERS ARE ALMOST THE PERFECT ARCHITECTURE FOR HP PROTEIN FOLDING

We can view Transformers as a type of Graph Neural Network (GNN) (Veličković (2023), Zhou et al. (2020)). The general form of a GNN is as follows. Given a graph $G = (V, E)$, with nodes $v \in V$ and edges $(u, v) \in E$,

$$m_v^{(l)} = \bigoplus_{u \in \mathcal{N}(v)} \phi_m(h_u^{(l-1)}, h_v^{(l-1)}, e_{uv}) \quad (1)$$

$$h_v^{(l)} = \phi_u(h_v^{(l-1)}, m_v^{(l)}) \quad (2)$$

where $h_v^{(l)}$ refers to the node features associated with node v at layer l , e_{uv} are edge features that are optionally present, ϕ_m is a learnable message function, ϕ_u is a learnable update function, $m_v^{(l)}$ is a generated message, $\mathcal{N}(v)$ are the neighbors of node v , and $\oplus$ is a permutation-invariant aggregation function. (1) is known as the message-passing step, and (2) is known as the update step. Yash

Transformers fit into this GNN blueprint with the message function ϕ_m being the attention mechanism, the permutation-invariant aggregation function $\oplus$ being addition, and the update function ϕ_u being the feed-forward network. The neighborhood of a node $N(v)$ defaults to all nodes in the graph, unless causal or attentional masking is present. Yash

The HP model can be framed as a graph, with the placement of the amino acid units as nodes and the protein backbone as edges connecting these nodes, we are left with graph data, on which we can apply a GNN. From the viewpoint that Transformers are just a type of GNN, and factoring-in the special properties of Transformer models discussed above, we gain two crucial advantages. Yash

Firstly, the Transformer model aggregates information across the entire placed sequence in a complex, non-linear manner due to the neighborhood being all nodes in the graph. This is far more powerful than applying other types of GNNs which work by aggregating only local information, and rely on a much less powerful global readout function $h_G = \rho(\{h_v^{(l)} | v \in V\})$. This is important for the HP protein folding problem as we must retain and use the context of the entire sequence laid-out so far to make optimal placement decisions. Yash

Secondly, the Transformer accounts for the sequential framing of the problem when position encodings are added as each step of the placement process is given an increasing positional embedding. In our experiments, we use a simple sin/cos scheme. Additionally, this naturally accounts for the presence of the protein backbone, which would otherwise have been lost due to the global attention mechanism. [Yash](#)

Due to these two reasons, Transformers are a great fit for this problem, however they have two key disadvantages. First is the quadratic cost of the attention mechanism, which would, in theory, dampen efforts to apply them to larger HP sequences, and second is the large amounts of data necessary to train the model, which results in longer convergence times. [Yash](#)

3.2.5 GRAPH NEURAL NETWORKS

Viewing the 2D HP protein folding problem (formalized in Section 4.1) from the lens of Geometric Deep Learning, we see that the domain Ω the data lives on is a lattice, with the folded protein backbone forming the signal $x : \Omega \rightarrow S_t$ on this lattice. [Yash](#)

From this perspective, and also drawing inspiration from similar work in material science and chemistry (Reiser et al. (2022)), a more natural representation of the problem is to view the folded protein as a graph, where the nodes are the amino acid units $\in \{H, P\}$, and the backbone connecting consecutive units are edges, making the two-dimensional underlying domain structure salient to the neural network. While sequence-based approaches (RNNs, Mamba, etc.) have to implicitly reconstruct the same two-dimensional representation from the one-dimensional sequence of actions taken, GNNs (Wu et al. (2020)) can directly operate on this graph data. [Yash](#)

We test the GAT (Veličković et al. (2017)) GNN architecture, which strikes a good balance between expressivity and computational complexity in the GNN family. [Yash](#)

3.3 TOWARDS NON-SEQUENTIAL HP PROTEIN FOLDING

When proteins fold in the real world, they do not do so in a sequential manner. All the atoms exert forces on each other simultaneously, resulting in the final structure. In the simplified HP model, this means that we can no longer use the relative actions/directions taken with respect to the previous amino acid unit as an input feature. Instead, we must model the two-dimensional position of each amino acid unit explicitly. In order to help lay the groundwork for future non-sequential work, we test a new state encoding (representation) (Figure 2, Section 4.8) with the Transformer architecture. [Yash](#)

3.4 EXPLORING DIFFERENT AGENTS—DDQN

A straightforward yet effective enhancement to DQN is the integration of Double DQN (Van Hasselt et al. (2016b)). This improvement addresses the issue of overestimation inherent in standard DQN by decoupling the selection of the action from its evaluation, which could potentially quicken convergence. [Tali](#)

3.5 REPLAY BUFFER

Another proposed improvement involves modifying the method by which experiences are sampled from the replay buffer. Intelligent sampling techniques, like prioritized experience replay, emphasize experiences that significantly influence learning, this includes scenarios with either high rewards or notable changes in the TD error (Schaul et al. (2015b)). This method ensures that important experiences are frequently learned, improving the efficiency of learning updates and helping the network focus on critical aspects of the environment faster. [Tali](#)

3.6 GENERALIZATION ACROSS HP SEQUENCES

As mentioned in the limitation statement, the reinforcement learning agent is trained upon one specific sequence which it learns to fold. Training on a single sequence can potentially lead to a highly optimized model for that specific sequence, achieving superior performance in predicting its structure, as well as gaining deeper insights into its specific dynamics. However, this may limit the model’s ability to generalize to other sequences, potentially restricting our understanding of the broader protein folding process. Moreover, this approach risks overfitting to the specific characteristics of the sequence used in training, thereby reducing the model’s effectiveness on new, unseen sequences. (Tali)

Another key improvement that the generalization may provide is using it as a pre-trained model for fine-tuning longer sequences. Adopting a pre-training approach on shorter sequences could potentially streamline the process, making it more efficient, especially for longer sequences which are typically more computationally demanding. This strategy could also mitigate the redundancy of training individual models for each sequence length. Given the absence of reported training times and the lack of experiments on significantly longer sequences (Chen and Huang (2005), documented training on sequences $N \leq 100$), we aim to explore whether a pre-training approach can effectively reduce training durations and enhance overall efficiency, as reported in Yu et al. (2019). Conversely, this approach may not be necessary if our enhanced neural network models, which discussed in Section 3.2, could inherently adapt to longer sequences without the incremental steps of pre-training and fine-tuning. (Tali)

In order to accommodate the introduction of variable sequence lengths in each episode, we will need to modify the state representation accordingly, ensuring that each episode dynamically adjusts to different sequence lengths. By doing so, we avoid processing an excess of no-action tokens and provide the LSTM model with a dynamically varying length of states, which it is inherently capable of handling. (Tali)

Furthermore, it is important to carefully consider the setup of rewards. Often, longer sequences naturally gather more rewards than shorter ones, which might lead to a preference within the model. This preference could distort the learning process, making the model disproportionately favor longer sequences just because of their length, and potentially missing easier wins that might be more achievable with shorter sequences. (Tali)

On the other hand, a preference for shorter sequences could reduce the model’s willingness to explore enough to effectively learn from longer sequences. To address these concerns, we will investigate different strategies for reward shaping. Since rewards significantly influence the learning process, disproportionately large or small rewards can cause fluctuations in updates. This variability might impact the stability and convergence of the learning algorithm. Effective reward shaping could help normalize the impact of sequence length on rewards, ensuring that the learning process remains balanced and focused on finding optimal strategies across all sequence lengths (Louie (2022)). An overview of the changes required to apply generalization is described in Section 4.9. (Tali)

An efficient way to learn in environments that include various lengths of sequences is by using curriculum learning (Narvekar et al. (2020)). Curriculum learning is a method inspired by the way humans often learn more effectively, starting with simpler or easier tasks and gradually moving to more complex ones. In our context of generalizing the environment, this could involve initially training a model on shorter, less complex sequences and progressively introducing longer or more complicated ones as the model’s performance improves. (Tali)

4 METHODOLOGY

4.1 ENVIRONMENT OVERVIEW

The HP protein folding problem is formulated as the agent performing a self-avoiding walk (SAW) on a two-dimension grid (lattice). We formulated the problem based on Yang et al. (2023) with some modifications. The problem starts with some HP sequence $\mathbf{S} \in \{H, P\}^N = (S_1, \dots, S_N)$ (where $N \in \mathbb{N}$ is the length of $\mathbf{S}$) for which we aim to find the

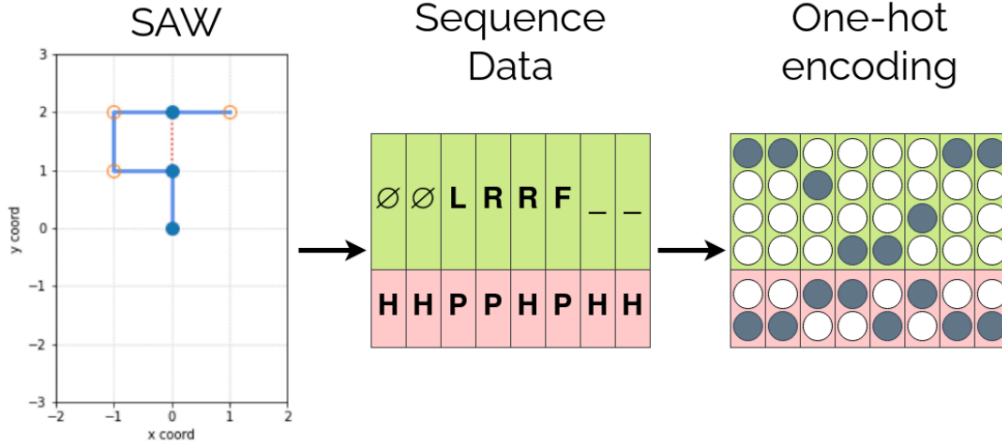


Figure 1: Left shows the incomplete peptide backbone laid out by the agent so far. Filled-in dots represent H s, and orange circles represent P s. The blue line connecting them represents the peptide backbone, with dotted orange lines representing H-H contacts. Center shows the state provided to the agent, with the actions taken on top, and the sequence on the bottom. Right is the one-hot encoded representation of the state. Figure from Yang et al. (2023). [Yash](#)

optimal two-dimensional folded structure. This HP sequence forms the peptide backbone of the protein. At every time step, the agent must place the next amino acid unit $S_t \in \{H, P\}$ in this sequence, such that it extends the backbone already placed on the lattice, while simultaneously satisfying the self-avoiding constraint. The SAW constraint can be formally stated as $\forall i, j \in [1, N]. i \neq j \Leftrightarrow (x_i, y_i) \neq (x_j, y_j)$. [Yash](#)

We formally denote placement of an amino acid unit as a function $p : \{H, P\} \rightarrow \mathbb{Z}^2$, where each amino acid unit S_t is associated with a two-dimensional coordinate. The goal of the agent is to find a folding for each amino acid unit S_t , which we denote $\mathbf{f} = (p(S_1), \dots, p(S_N))$, such that the energy of the folded structure (E_f) is minimized. This folding must also be contiguous (so that adjacent amino acid units in the sequence occupy adjacent cells on the lattice), and must satisfy the self-avoiding constraint. [Yash](#)

A contact is formed when two amino acid units S_m and $S_n, m \neq n$ are placed on adjacent lattice grid sites, but are not adjacent in the HP sequence $\mathbf{S}$, ($S_n \notin \{S_{m-1}, S_{m+1}\}$). This means that the two units are not directly linked by the peptide backbone. A contact between two H units is an H-H contact. Each H-H contact in the folded sequence reduces the energy by 1. The goal is to find the placement $\mathbf{f}$ minimizing the energy (or equally maximizing the number of H-H contacts, done in practice). [Yash](#)

An overview of the environment can be found in Figure 1, with a formal definition of the problem in Section 4.5. [Yash](#)

4.2 ACTION-SPACE

Ensuring contiguity of the placement $\mathbf{f}$ is achieved by carefully choosing the action-space of the agent. The action-space of the agent is a scheme relative to the agent's current direction of travel, where the agent, at every time step, can choose to turn left (L), turn right (R), or continue travelling forwards (F). As an example, suppose the agent was travelling eastwards (e.g. from coordinates $p(S_{t-2}) = (5, 0)$ to $p(S_{t-1}) = (6, 0)$), and has to place the next amino acid unit S_t . The following would be the resulting coordinates for S_t given the actions $A_t \in \{L, F, R\}$ taken by the agent:

$$p(S_t) = \begin{cases} (6, 1) & \text{if } A_t = L \\ (7, 0) & \text{if } A_t = F \\ (6, -1) & \text{if } A_t = R \end{cases} \quad (3)$$

Through choosing an action-space that is relative to the current direction of travel, we ensure that the peptide backbone remains contiguous, as well as making detection of self-avoiding violations trivial as we simply check if a token was previously placed at the same coordinate: $p(S_t) \in \{p(S_n) \mid n \in [1, t)\}$. [Yash](#)

Note that the energy of the folded protein f is invariant to translations and rotations of $n\frac{\pi}{2}, n \in \mathbb{Z}$ radians on the lattice. To ensure model capacity and training time is not wasted on such transformations of the folded protein, we canonicalize the problem by requiring the layout to begin at the origin ($p(S_1) = (0, 0)$), travel forwards for the second timestep ($p(S_2) = (0, 1)$), and force the first turn $\in \{L, R\}$ made by the agent to always be a left turn (L). [Yash](#)

4.3 STATE-SPACE

At every timestep, the state s_t provided by the environment to the agent consists of two fixed-length vectors (of N elements each) of the actions taken for each S_t , and the sequence to be laid out.

$$\begin{aligned} s_t &= (\mathbf{A}, \mathbf{S}) \\ \mathbf{A} &= \bar{A}^N \\ \bar{A} &= \{L, F, R, \emptyset\} \\ \mathbf{S} &= \{H, P\}^N \end{aligned} \quad (4)$$

$\bar{A}_t$ is the action taken by the model while placing S_t , with $\emptyset$ signifying that the model is yet to take an action for that amino acid unit. Note that $\bar{A} \neq A$, since the agent cannot take the nothing action $\emptyset$. Also, $\bar{A}_1 = \emptyset \wedge \bar{A}_2 = \emptyset$ by convention due to the layout canonicalization discussed in the previous section. [Yash](#)

The environment does not provide a direct view of the underlying lattice, but this can be reconstructed by the agent, if desired. Such a state representation is memory-efficient and all that is required by neural networks for sequence processing such as RNNs for performing this task. The state provided by the environment is also not one-hot encoded, but is instead the responsibility of the machine learning model. [Yash](#)

Since the model is trained separately on each HP sequence, the sequence length N is constant throughout the training process. Before providing the state to the machine learning model we apply one-hot encoding $g : s_t \rightarrow \mathbb{B}^{6 \times N}$ where $\mathbb{B} = \{0, 1\}$. For each HP component S_t , the first four rows hold the one-hot encoding for $\bar{A}_t$. The last two rows indicate whether it is an H or P unit (see Figure 1). [Tali](#)

4.4 REWARDS

The learning environment is a sparse-reward setting with a reward equal to the number of H-H contacts only provided at the end of the episode (when the complete sequence $\mathbf{S}$ has been successfully laid out, without violating the SAW constraint). Upon violation of the SAW constraint, the episode is terminated immediately with 0 reward, enforcing the constraint that final laid-out sequences are self-avoiding. [Yash](#)

4.5 FORMALIZING THE HP PROTEIN FOLDING PROBLEM AS A MARKOV DECISION PROCESS [TALI](#)

We can define the problem as an MDP specified by the tuple $M = \langle \bar{S}, A, P, \gamma, R \rangle$. $\bar{S} = \{s_t \mid t \in [1, N]\}$ is the set of all possible states (see section Section 4.3). At time t , the agent

given state s_t , takes action $a_t \in A$ in order to place the next HP amino acid unit on the two-dimensional lattice, and deterministically moves to s_{t+1} . After that, the agent receives a reward $r_{t+1} \in \mathbb{R}$ based on the reward function R .

The reward function R is defined as follows:

$$r_{t+1} = \eta \cdot |E_{f[t]}| \quad (5)$$

$$\eta = \begin{cases} 1 & \text{if episodes is finished, i.e. } t = N \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

$0 \leq \gamma \leq 1$ is the discount factor, which represents the importance of future rewards compared to immediate rewards. Yang et al. (2023) set it to 0.98.

P is the transition probability distribution $P(s_{t+1}|s_t, a_t)$ which is deterministic in this model.

4.6 BASELINE TALI

Following Yang et al. (2023), we implemented the following methods:

- Training agent: Deep Q-Learning (DQN), with separate target and policy networks and ε -greedy for exploration and exploitation trade-off, with exponential ε decay.
- Model: Long Short Term Memory (LSTM). Yang et al. (2023) argued that this is the reason to reach high performance. The intuition to this is that the LSTM captures long range interactions among the HP sequence presented. Hidden size is switched between 64, 256, 512 (depending on N and computation limitations).
- Loss and training: Using Huber loss, which is more resilient to outliers than Mean Squared Error (MSE), for better optimizing the Q-functions. Trained on mini-batch size 32, with the Adam optimizer and learning rate of 0.0005.
- Replay buffer: size is defined by $\min(50000, \frac{\#episodes}{10})$, and sampled randomly.
- The benchmark HP sequences (whose ground truth minimum energies are known) are from Istrail’s research group (no reference has provided—see Table 1 in Yang et al. (2023)).

4.7 OVERVIEW OF MODEL ARCHITECTURES TESTED YASH

In this work, we explore several alternatives to the LSTM-based setup. Note that we also include a reference LSTM, with hyperparameters replicated from Yang et al. (2023) as a baseline to compare against. We tune our models to match the total computation time of the baseline for fairness as closely as possible. The model architectures tested are:

Model Architecture	Number of Layers	Hidden Dimension Size
Reference LSTM	2	256
Mamba Selective State Space Model	3	32
Transformer	2	128
Masked Transformer	2	128
Graph Attention Network	3	32

The Masked Transformer is a special version of the transformer which is only allowed to attend to adjacent tokens. This is achieved by the use of a causal mask. The Masked Transformer thus propagates information in a very similar fashion to GATs, which can only communicate via the protein backbone. Additionally, the Masked Transformer, like

with GAT, is also given a summation pooling layer so that information can be aggregated across the entire HP sequence. This is due to the fact that causal masking (or propagation via the protein backbone in GAT) limits the receptive field of the neural network to only local interactions. The addition of the graph pooling layer increases the expressive power of the neural network in both of these cases (Bianchi and Lachi (2023)). Details of these architectures are illustrated in Figure 3. [Yash](#)

4.8 OVERVIEW OF THE COORDINATE ENCODING SCHEME

We explore the use of the absolute lattice coordinates as an alternative encoding of the state. Each amino acid unit is represented by a node, with the node feature being its two-dimensional position $\in [0, 1]^N$ (normalized by dividing by the length of the HP sequence to ensure floating point numerical stability) and a one-hot encoding of whether it represents an H or a P . An illustration detailing this can be found in Figure 2. [Yash](#)

4.9 GENERALIZATION SCHEME

To introduce generalization capabilities, we modified the environment and the model in several ways: [Tali](#)

1. **Sequence Variability:** At each environment reset, we introduce a new sequence, which could either be different or the same as the previous epoch, depending on the learning strategy. [Tali](#)
2. **Reward Normalization:** To ensure reward consistency across different sequences and more stable training, we normalize rewards before updating the policy. We apply z-score normalization to the sampled trajectories, similar to Deng et al. (2022). The z-score for rewards is computed as:

$$\text{Z-score}(s, t, \tau_i) = \frac{r_t^i(s) - \mu(\tau)}{\sigma(\tau)}$$

Where $\tau = \{\tau_1, \tau_2, \dots, \tau_m\}$ represents the set of accumulated trajectories before updating the policy through policy optimization, where each τ_i consists of a sequence of states and rewards. We define $r_t^i(s)$ as the reward received at state s at time step t in trajectory τ_i . $\mu(\tau)$ and $\sigma(\tau)$ are the mean reward and standard deviation of the reward across all trajectories. Consequently, we adapted the LSTM architecture to handle masking, allowing it to process padded sequences. [Tali](#)

3. **Handling Variable Sequence Lengths:** We introduced a maximum sequence length, ensuring that the trajectory buffer maintained a consistent size. [Tali](#)

5 EXPERIMENTS

The majority of our experiments in this study focused on the 20mer-B HP chain, which has an optimal value of 10. For reference to these experiments, random sampling results are illustrated in Figure 4. [Tali](#)

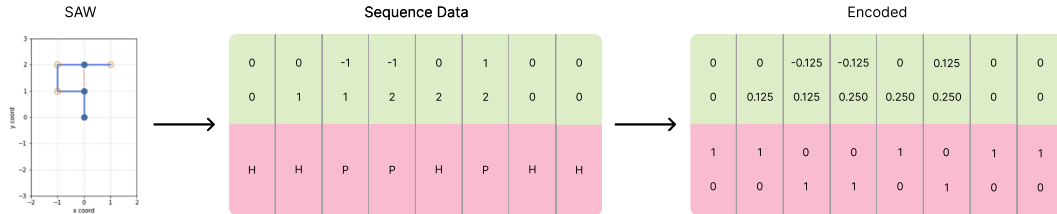


Figure 2: An illustration of what the new input data based on the two-dimensional positions of the amino acid units looks like for the same incomplete peptide backbone laid-out so far as in Figure 1. [Yash](#)

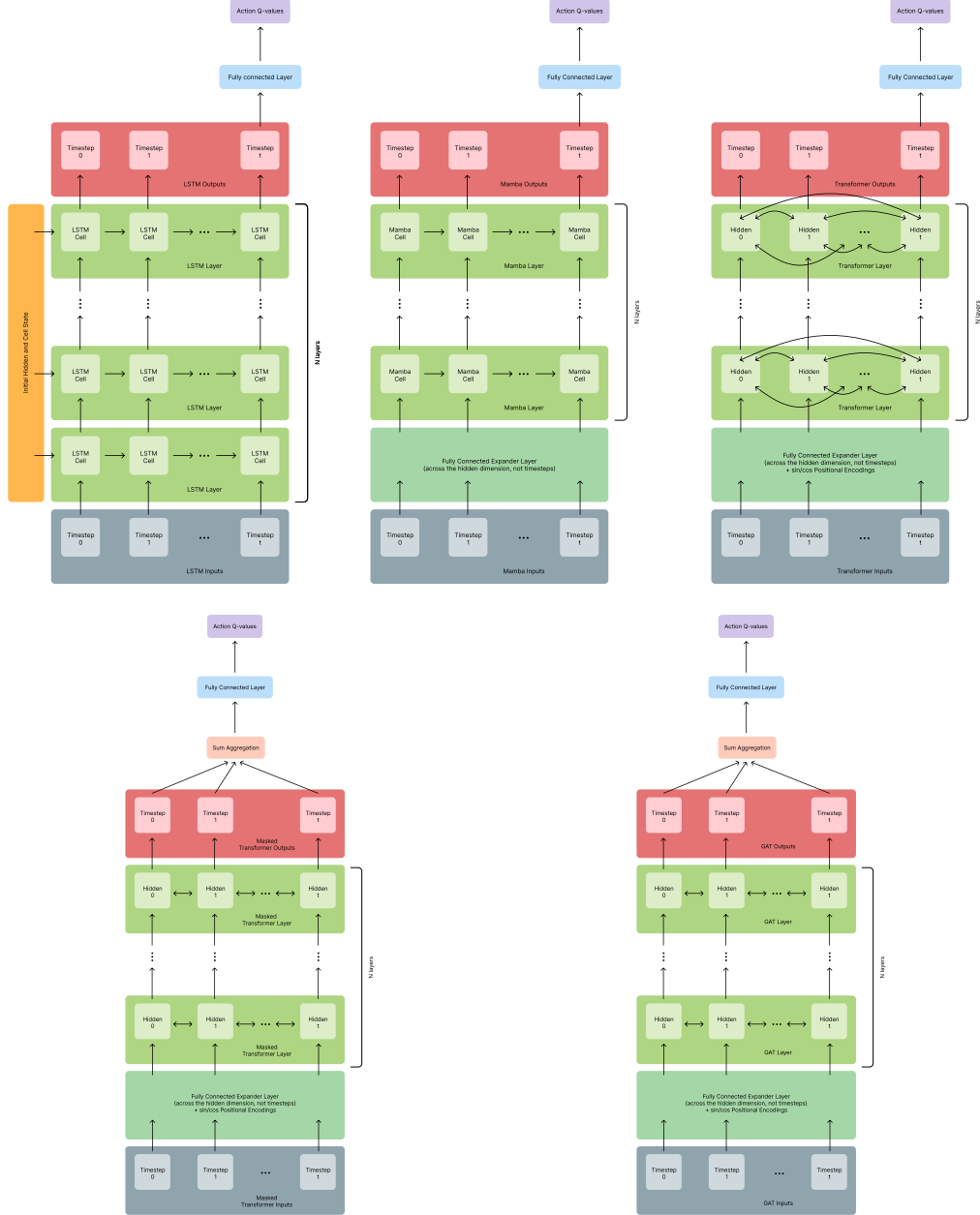


Figure 3: An overview of the different model architectures tested for the Q-function. *Top row: Left* shows the LSTM architecture used by the original authors (Yang et al. (2023)). *Center* shows our Mamba-based architecture. Note the expander layer present to map the inputs into the hidden dimension’s size, which is implicitly handled by the LSTM. *Right* shows our Transformer-based architecture. Note the presence of sin/cos positional encodings to break the Transformer’s permutation equivariance property. *Bottom row: Left* is the Masked Transformer that simulates the way information is propagated in GAT. *Right* is the GAT-based architecture. Both of these architectures use the expressive sum pooling to create the graph-level feature that is then used by the fully-connected layer to make Q-value predictions. Yash

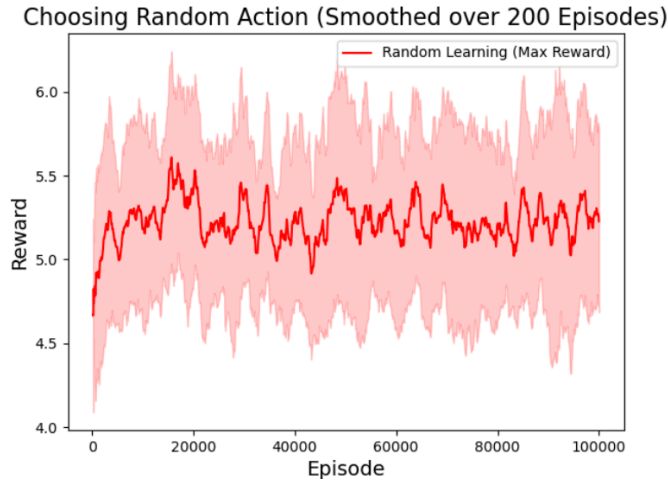


Figure 4: The plot shows the maximum reward over a 200-episode window using random action sampling, focusing on the 20mer-B sequence. The values averaged and with standard deviation, calculated from three different seeds: 42, 0, and 123. Tali Yash

5.1 PPO ACHIEVES STATE-OF-THE-ART PERFORMANCE

In this experiment, we implemented the PPO algorithm. Conducting experiments on the 20mer-B sequence with an LSTM network required extensive hyperparameter tuning. This aligns with findings from (Engstrom et al. (2019)), which highlight that gradient-based updates in policy optimization methods make them highly sensitive to hyperparameters, significantly influencing learning dynamics. Tali

To improve stability, we implemented an adaptive learning rate decay, where every 3,000 episodes, the learning rates of both the actor and critic networks were scaled down to 70% of their current values (starting from 0.0005 and 0.001 respectively), with a lower bound of $1e^{-5}$. Additionally, we set the entropy loss coefficient to 0.04, which prevents policies from becoming overly deterministic too early and encourages continued exploration. To ensure stable learning, we applied a clipping parameter $\varepsilon = 0.2$, which limits drastic updates to the policy and helps prevent divergence. We also added gradient clipping as we saw that learning is not stable. Tali

We found that one of the most critical hyperparameters affecting convergence was the number of trajectories collected before updating the policy, as well as the number of epochs used during training. For 20mer-B, we found that collecting 300 trajectories before an update and training for 150 epochs produced the best results. Collecting fewer than 300 trajectories led to faster convergence, but performance across episodes became more erratic. Meanwhile, when the number of training epochs was either too low or too high, training exhibited greater fluctuations, likely due to underfitting, where the policy does not sufficiently learn from experience, or overfitting, where the policy becomes too specialized to certain experiences and fails to generalize well. With this setup, PPO successfully converged to the highest reward (10) in approximately 4,000 episodes, which is five times faster than DQN-based methods, requiring only 10 minutes of runtime, see Figure 5. Tali

As illustrated in Figure 6, when rewards are not smoothed, the PPO method exhibits less stability compared to DQN, particularly after reaching optimal values. This instability can be attributed to the nature of PPO as an on-policy algorithm, which depends solely on current data. While it employs a clipped objective function to limit the magnitude of policy updates, it is still vulnerable to sudden policy shifts when environmental conditions change. Conversely, DQN benefits from a replay buffer that allows it to revisit diverse experiences, promoting a more balanced learning process and consistent convergence. Additionally,

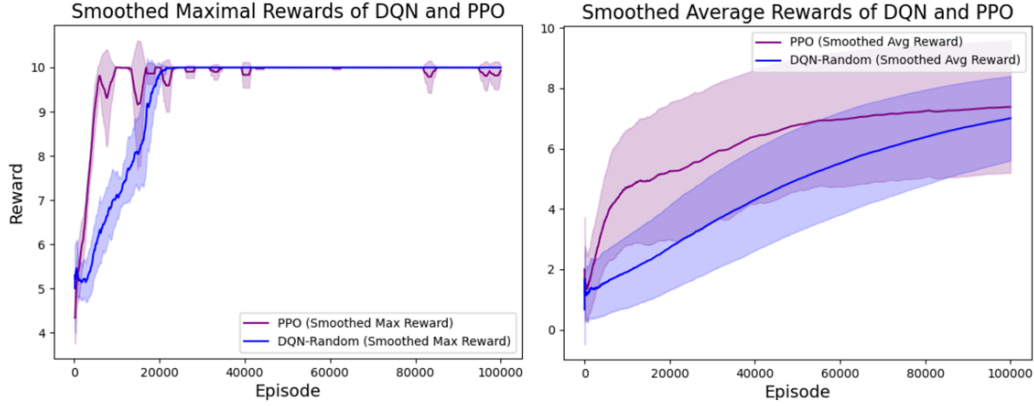


Figure 5: The left plot displays the smoothed maximum reward over a 200-episode window, while the right plot presents the average reward, smoothed over 10,000 episodes, averaged and with standard deviation, calculated from three different seeds: 42, 0, and 123. These visualizations track the reward progression for PPO (in purple) and DQN with a random replay buffer (in blue), specifically focusing on the 20mer-B sequence. PPO demonstrates significantly faster convergence, converging to the maximum reward of 10 within approximately 6,000 episodes, whereas DQN requires around 20,000 episodes to converge. (Tali)

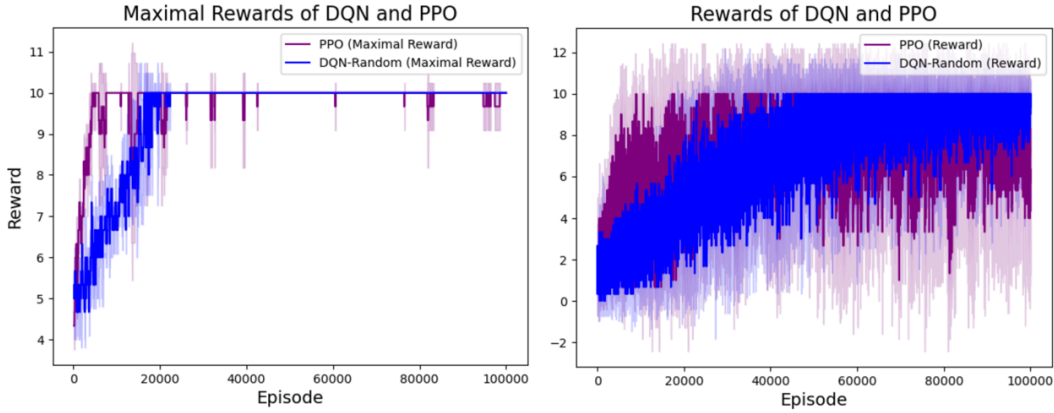


Figure 6: The left plot showcases the maximum reward achieved over a 200-episode window, and the right plot displays the per-episode reward, averaged and with standard deviation, calculated from three different seeds: 42, 0, and 123. These visualizations track the reward progression for PPO (in purple) and DQN with a random replay buffer (in blue), specifically focusing on the 20mer-B sequence. (Tali)

DQN employs a separate target network, which helps mitigate the moving-target bias and stabilizes training. (Tali)

We applied the same hyperparameter settings to multiple sequences to explore whether we could achieve faster convergence across different sequences. However, when testing 25mer, for instance, we found that the optimal reward of 8 was never reached, even after 350K episodes, with the highest recorded reward being 7 (see Table 1). To address this, we increased the number of trajectories collected before each update to 500 and raised the number of training epochs to 200. With these adjustments, convergence accelerated and was achieved in 25K episodes, compared to 45K episodes using the baseline DQN. This result underscores the importance of sequence-specific hyperparameter tuning. More complex sequences may require larger batches of collected experience and more training iterations per update to learn optimal policies effectively. Therefore, for the generalized environment experiment (see Section 5.9), the pre-trained sequences were trained on their matching hyperparameters. (Tali)

Table 1: This table compares the episodes required for convergence using DQN and PPO algorithms, alongside the optimal rewards for various sequences. The third and fourth columns detail the initial and fine-tuned convergence episodes for PPO, including adjustments in the number of trajectories collected and optimization epochs (shown in the fifth and sixth columns). Entries marked with an asterisk (*) denote sequences where convergence was not achieved, specifying the maximum number of episodes completed and the highest reward obtained. (Tali)

Sequence Name	Conv. epoch (DQN)	Conv. epoch (PPO)	Conv. epoch (PPO + FT)	#Trajectories (M)	#Epochs (K)	Optimal Value
20mer-A	20K	-	3K	300	150	9
20mer-B	20K	-	6K	300	150	10
24mer	27K	35K	15K	500	200	9
25mer	45K	350K*/7	25K	500	200	8
36mer	100K	100K	350K*/13	1000	200	14
48mer	350K*/21	350K*/19	350K*/21	1000	200	23
50mer	200K	350K*/17	350K*/18	1000	200	21
100mer	-	-	350K*/32	1000	200	50

Table 1 displays the results of experiments comparing the PPO and the baseline DQN methods, as applied in (Yang et al. (2023)). Detailed components of each sequence’s H-P are available in Yang et al. (2023) and Chen and Huang (2005). For sequences shorter than 36, PPO required fewer epochs to converge than DQN, particularly when adjusting the hyperparameters mentioned. For longer sequences, despite numerous attempts with various hyperparameter combinations, optimal convergence often remained elusive (indicated by an asterisk * in Table 1), and in some cases, we were unable to replicate the results from (Yang et al. (2023)). This discrepancy can be attributed to several factors.

Firstly, for sequences of length $N \leq 36$, we matched the hidden layers and sizes specified in the referenced study. However, for longer sequences, computational limitations necessitated a reduction to a hidden size of 64 across three layers. Additionally, updates to the PPO were executed in minibatches due to the same constraints. Secondly, adjustments were only made to two hyperparameters due to the high cost of experimenting with longer sequences, preventing further exploration of other critical parameters such as learning rate, batch size, clipping coefficient, etc. We expect that with fewer computational constraints, improved results might be achievable for longer sequences. (Tali)

5.2 OVERVIEW OF DIFFERENT MODEL ARCHITECTURE PERFORMANCE (YASH)

The following table summarizes the performance of the different model architectures tested:

Table 2: The encoding refers to whether the original (Figure 1) relative encoding scheme is used, or the (Section 4.8) coordinate encodings. The optimal epoch refers to the epoch for the best seed where the highest known number of H-H contacts for the 20-mer sequence (HHHPPHPHPHPHPHPHPH), 10, is found. The convergence epoch refers to when training stabilizes for the best, where the model consistently achieves the optimal folding. The best found refers to the folding with the maximum number of H-H contacts actually found by the best performing seed.

Model Architecture	Encoding	Optimal Epoch	Convergence Epoch	Best Found
Reference LSTM	Relative	15k	20k	10
Mamba	Relative	7k	15k	10
Transformer	Relative	17k	25k	10
GAT	Relative	76k	-	10
Masked Transformer	Relative	-	-	9
Transformer	Coordinate	36k	50k	10

5.3 MAMBA ACHIEVES STATE-OF-THE-ART PERFORMANCE

The current problem formulation models the HP protein folding problem as a sequential decision making problem, and thus successfully storing and retrieving the context of actions taken many timesteps earlier in the episode is essential to finding good solutions. We found that Mamba offered significantly improved performance (Figure 8, Table 2) over the LSTM baseline (Figure 7, Table 2). From the top graph in both figures, we see that on the 20-mer test sequence, Mamba is able to achieve the optimal folding (10 H-H contacts) around 8,000 episodes earlier than the previous LSTM-based method, which required a total of 15,000 episodes on the best performing seed. Mamba achieves this superior performance consistently across all three seeds, achieving the optimal folding in a total of 11,000 episodes on the worst performing seed (still 4,000 episodes before the most optimistic LSTM case). Additionally, the bottom graphs on both figures show that Mamba is able to reach the stable converged state around 10,000 episodes before LSTM. (Yash)

While Mamba achieves great performance, it suffers from more instability after convergence than LSTM. For example, there is noteworthy dip in performance around episode 60,000 for one of the seeds (colored blue in the top graph in Figure 8). Interestingly, these correspond almost perfectly with spikes in the Q-function loss (middle graph in Figure 8). While we attempted to mitigate this issue with the use of gradient clipping, this unfortunately did not alleviate this issue. The LSTM model is also not immune to these performance fluctuations, but seems more resilient than Mamba. (Yash)

5.4 TRANSFORMER PRODUCES UNREMARKABLE RESULTS

The Transformer-based model (results in Figure 9, Table 2) does not always converge to the optimal folding across all the runs, and its convergence rate is similar to that of the baseline LSTM. The Q-network loss may explain why this is the case. The middle graph in Figure 9 shows that although the loss falls with more episodes, there are still numerous spikes suggesting that the model is still actively learning from new data. Transformers are notorious for requiring a significantly large amount of data to train, even on simple algorithmic tasks (Ruoss et al. (2023)), so it is perhaps unsurprising that this is the case here. We postulate that the Transformer, given sufficient exploration time, can outperform the other models on long HP sequences, as we show that managing long-range dependencies is crucial for this task in the next section. (Yash)

5.5 THE FAILURE OF GAT AND THE IMPORTANCE OF LONG-RANGE DEPENDENCIES

The GAT-based model operates on input graph data composed of each of the amino acid units of the HP sequence as nodes, and edges connecting only adjacent units. Unlike recurrent networks like LSTM and Mamba that propagate a hidden state throughout the entire input sequence, with the final hidden state used to make the action prediction, information in GAT is propagated in parallel between nodes through neural message passing. (Yash)

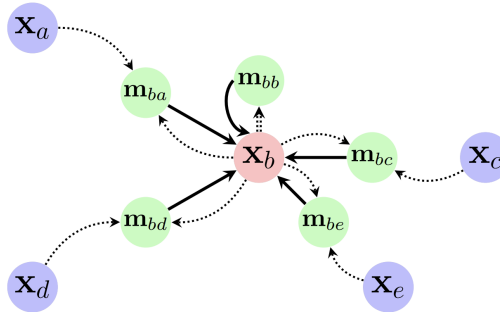


Figure 10: An illustration of neural message passing showing the local information propagation around node x_b . Diagram from Veličković (2022). (Yash)

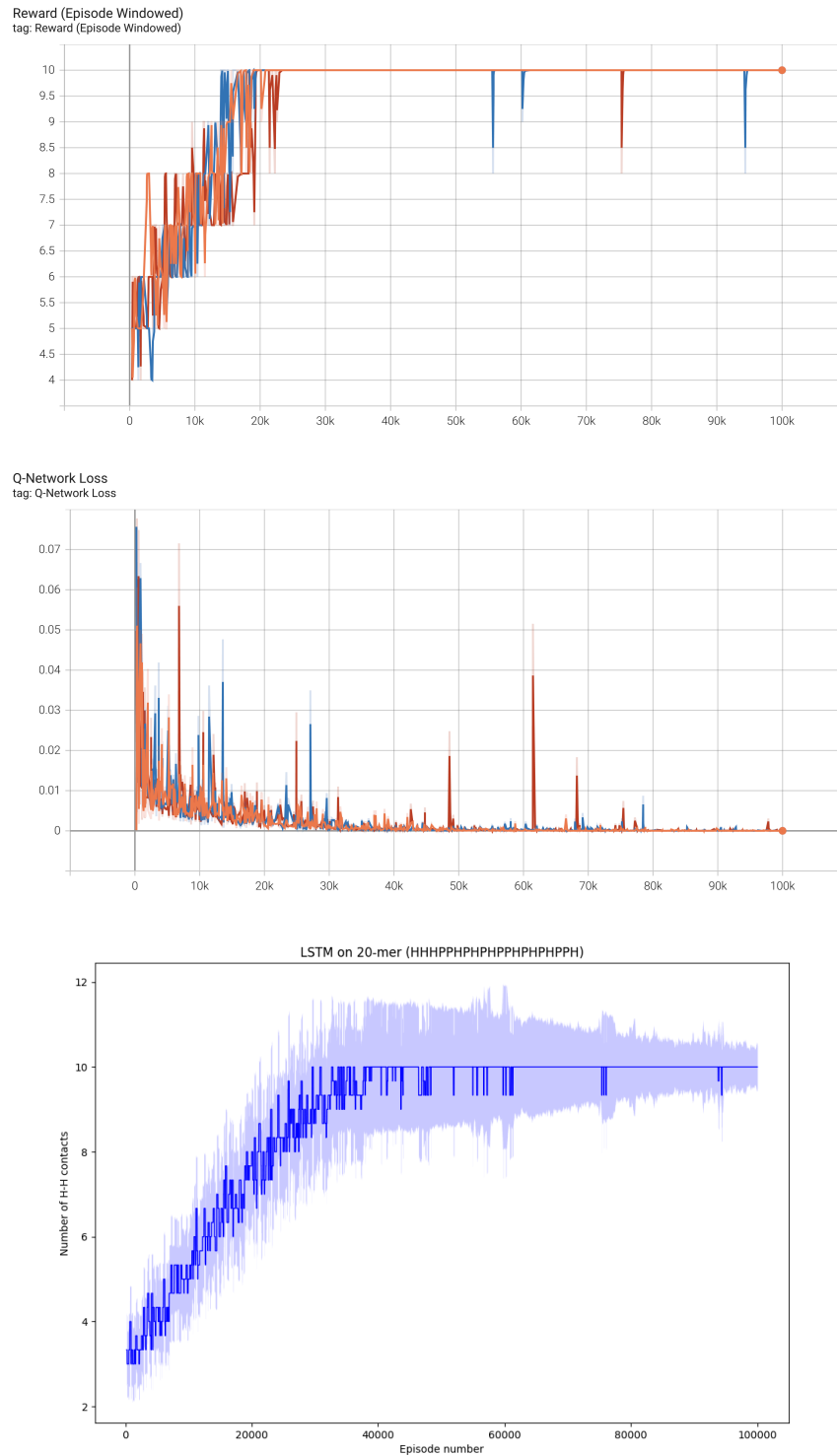


Figure 7: This figure shows the performance of the baseline LSTM model on the test 20-mer sequence (HHHPPHPPHPPHPPHPPH). *Top*: The maximum reward received over a rolling 200-episode window by each of the three seeds (1, 2, 3) tested. *Middle*: The Q-Network loss for each seed. *Bottom*: The maximum reward received over a rolling 200-episode window over the average of the three seeds. The shaded area represents 1 standard-deviation above and below the mean. Yash

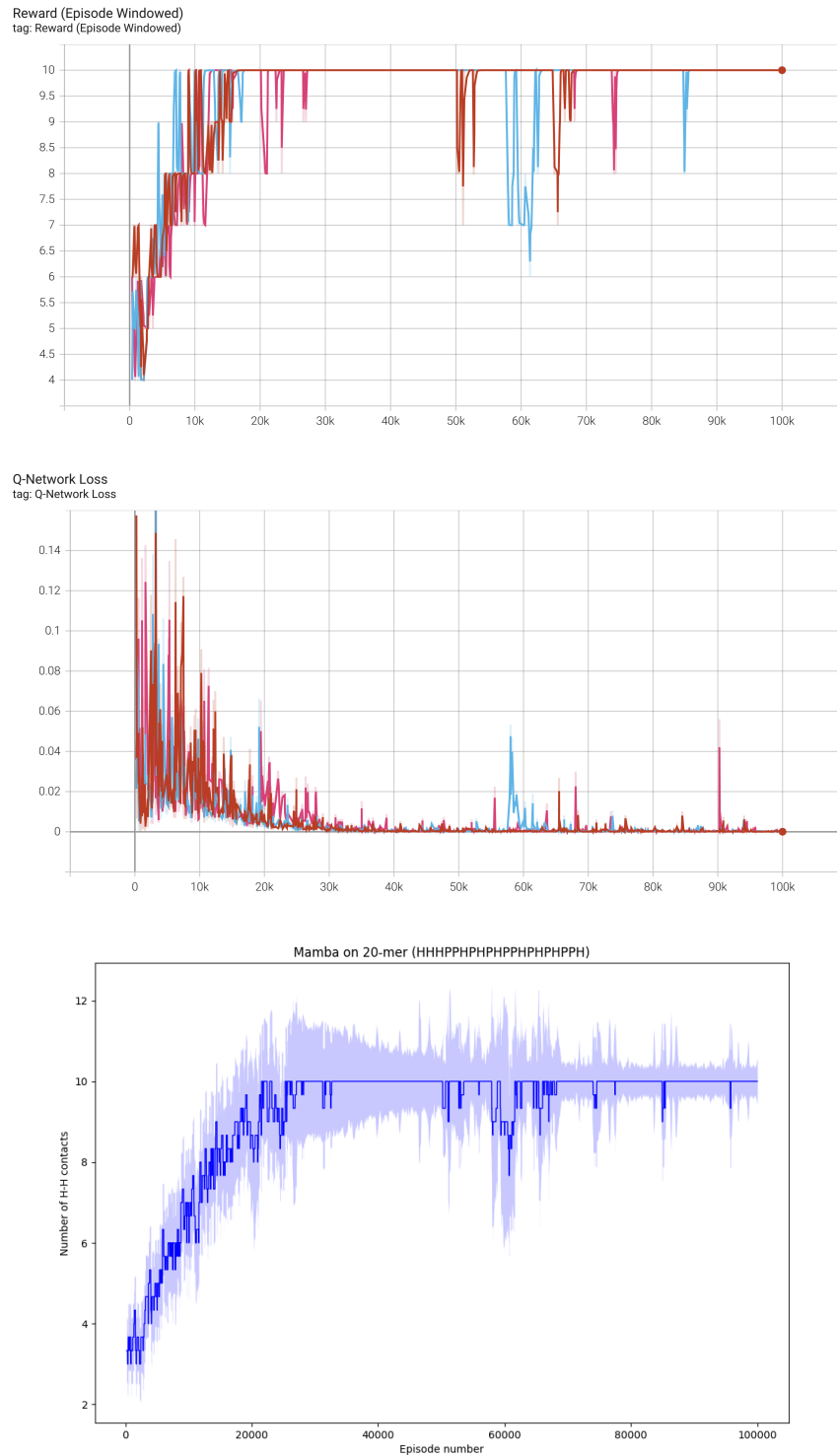


Figure 8: This figure shows the performance of the Mamba model on the test 20-mer sequence (HHHPPHPPHPPHPPHPPH). *Top:* The maximum reward received over a rolling 200-episode window by each of the three seeds (1, 2, 3) tested. *Middle:* The Q-Network loss for each seed. *Bottom:* The maximum reward received over a rolling 200-episode window over the average of the three seeds. The shaded area represents 1 standard-deviation above and below the mean. Yash

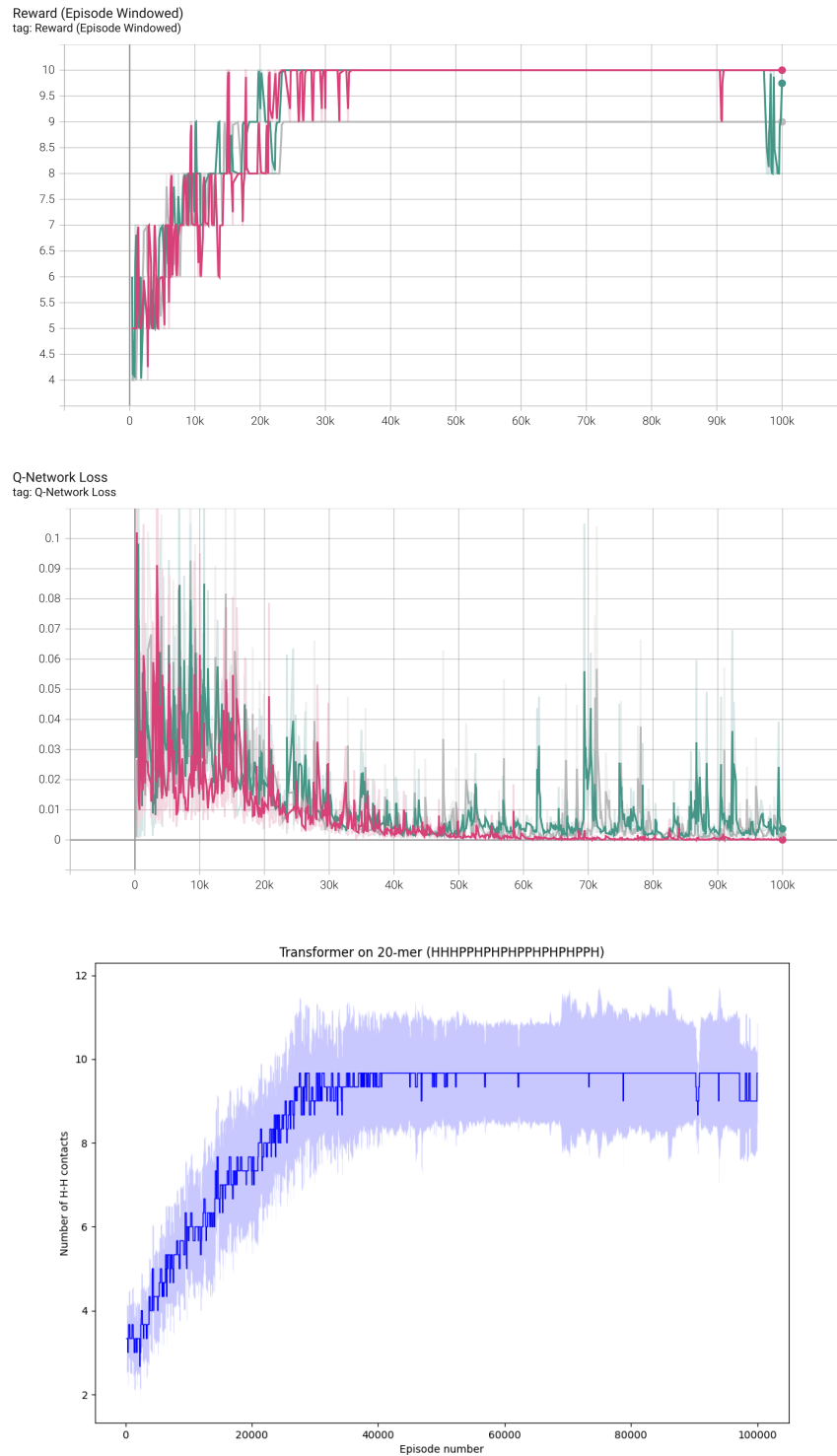


Figure 9: This figure shows the performance of the Transformer model on the test 20-mer sequence (HHHPPH-PHPHPHPHPHPPPH). *Top*: The maximum reward received over a rolling 200-episode window by each of the three seeds (1, 2, 3) tested. *Middle*: The Q-Network loss for each seed. *Bottom*: The maximum reward received over a rolling 200-episode window over the average of the three seeds. The shaded area represents 1 standard-deviation above and below the mean. Yash

Pairwise message passing between nodes, along the edges, forms the fundamental primitive of most GNNs. This pairwise interaction makes GNNs operating on sparsely connected graphs (like here) scalable to large graphs as all interactions are local to a node (Veličković (2022)) (within its one-hop neighborhood $\mathcal{N}_u = \{v \in \mathcal{V} | (v, u) \in \mathcal{E}\}$ for some graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$), unlike models such as Transformers that implicitly assume a fully connected graph. However, this limits the maximum distance at which information can be aggregated from for a given number of message-passing steps (or alternatively, GNN layers). This maximum distance is known as the GNN’s receptive field, and this problem is known as the under-reaching problem (Lu et al. (2024)) [Yash](#).

In our tests, we used a GAT with 3 layers, and thus a receptive field of only 3 hops. To help the network capture information from a further distance away (to have memory of where the network has placed previous H s and P s), we thus add a graph-level pooling layer. It is important to note that this graph-level pooling to aggregate information is less expressive than the message passing steps, and so cannot completely substitute the need for a larger receptive field in problems that require it. [Yash](#)

Unfortunately, HP protein folding is a task where this larger receptive field is necessary. We see from the results in Figure 11 and Table 2, that the GAT-based model performs barely better than random (Figure 4), only ever achieving the optimal folding for one of the seeds, with network behavior never stabilizing. Additionally, the Q-network loss does not decrease, suggesting that the model is failing to learn. [Yash](#)

In order to confirm that the lack of long-range dependency modelling capability is key in preventing GAT from working, we took the Transformer model (shown previously to work) as it can almost be viewed as a GAT operating on a fully-connected graph (alleviating this issue), and masked its computed attention matrices so tokens could only attend to adjacent tokens. This simulates the local information propagation of GAT using a Transformer—a model we know (Figure 9, Table 2) is capable of solving this problem. We see from the results in Figure 12 and Table 2, that this Masked Transformer is unable to solve the problem too. We believe that this experiment provides strong evidence that this problem requires models that are capable of good modelling of long-range dependencies. [Yash](#)

5.6 TRANSFORMER FINDS IT HARD TO USE COORDINATE ENCODINGS

As discussed in Section 3.3, testing the use of Coordinate Encodings (Section 4.8) is important to lay the foundation for non-sequential RL-based HP protein folding approaches. We tested the use of these encodings with the Transformer model, as it is a type of GNN, and has great long-range dependency modelling capabilities (in our testing Coordinate Encodings did not work with GAT). As shown by the results in Figure 13 and Table 2, the Coordinate Transformer (which is simply the Transformer working with Coordinate Encodings) is able to get the optimal result in some runs, but does not produce outstanding results. Nonetheless, the runs produce stable results (unlike Mamba, for instance), despite spikes in the Q-network loss, and so offer a promising direction for future work. [Yash](#)

We posit that the principal limitation of Coordinate Encodings is that they are an absolute positional encoding scheme (from the Transformer literature). This means that Transformers lose their translation invariance property, which may limit a model’s ability to recognize similar patterns appearing at different positions in the HP sequence. Replacing the Coordinate Encodings with a Rotary Coordinate Encoding scheme (inspired from Su et al. (2024)) may help alleviate this issue. [Yash](#)

5.7 EQUIVALENCE IN EFFICACY—COMPARING DDQN TO DQN

To compare the performance of different agents, we maintained the same hyperparameter values as those specified in the baseline to ensure a fair comparison. The models were trained for 100,000 episodes. [Tali](#)

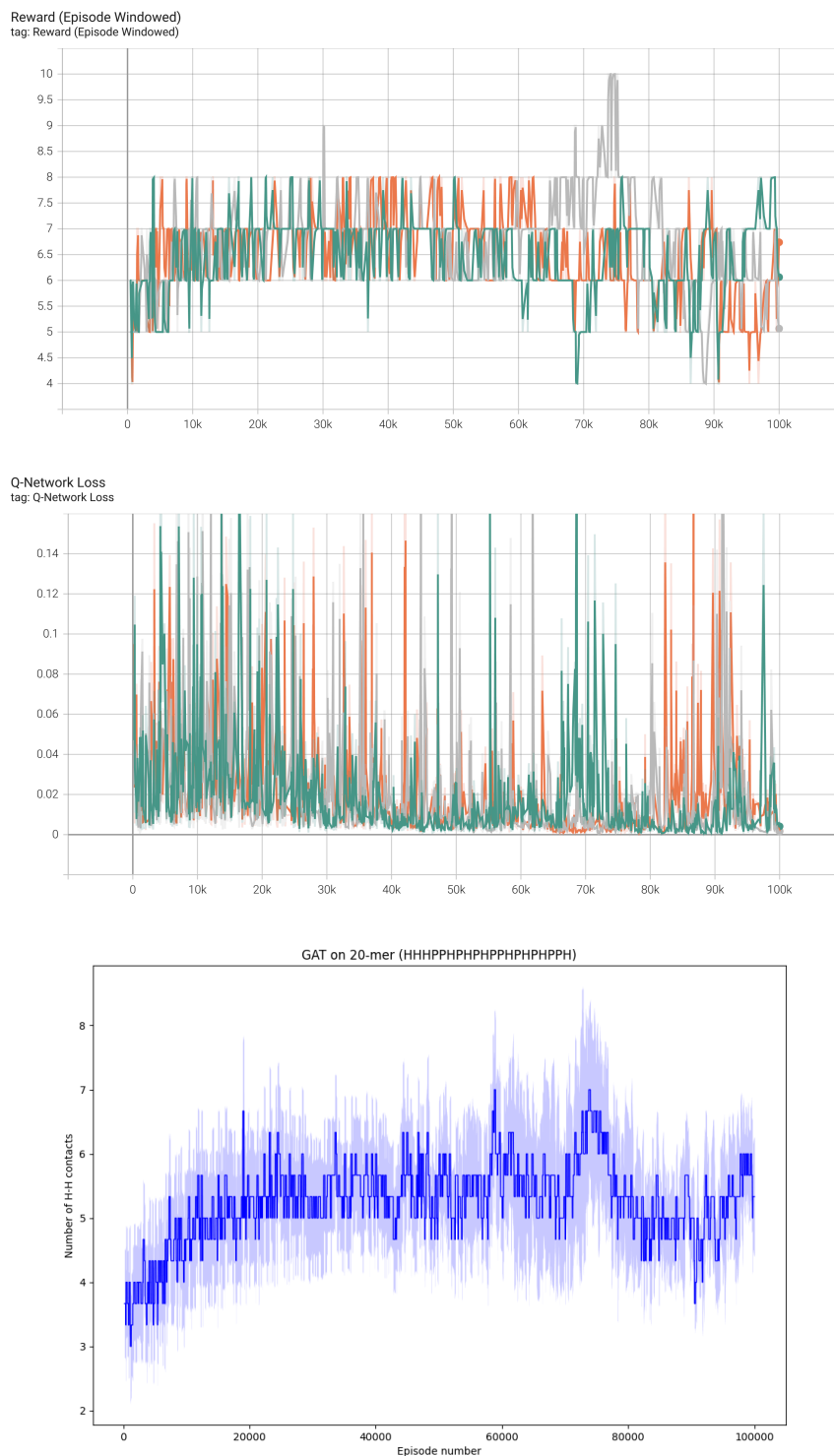


Figure 11: This figure shows the performance of the GAT model on the test 20-mer sequence (HHHPPHPHPHPHPHPHPH). *Top:* The maximum reward received over a rolling 200-episode window by each of the three seeds (1, 2, 3) tested. *Middle:* The Q-Network loss for each seed. *Bottom:* The maximum reward received over a rolling 200-episode window over the average of the three seeds. The shaded area represents 1 standard-deviation above and below the mean. Yash

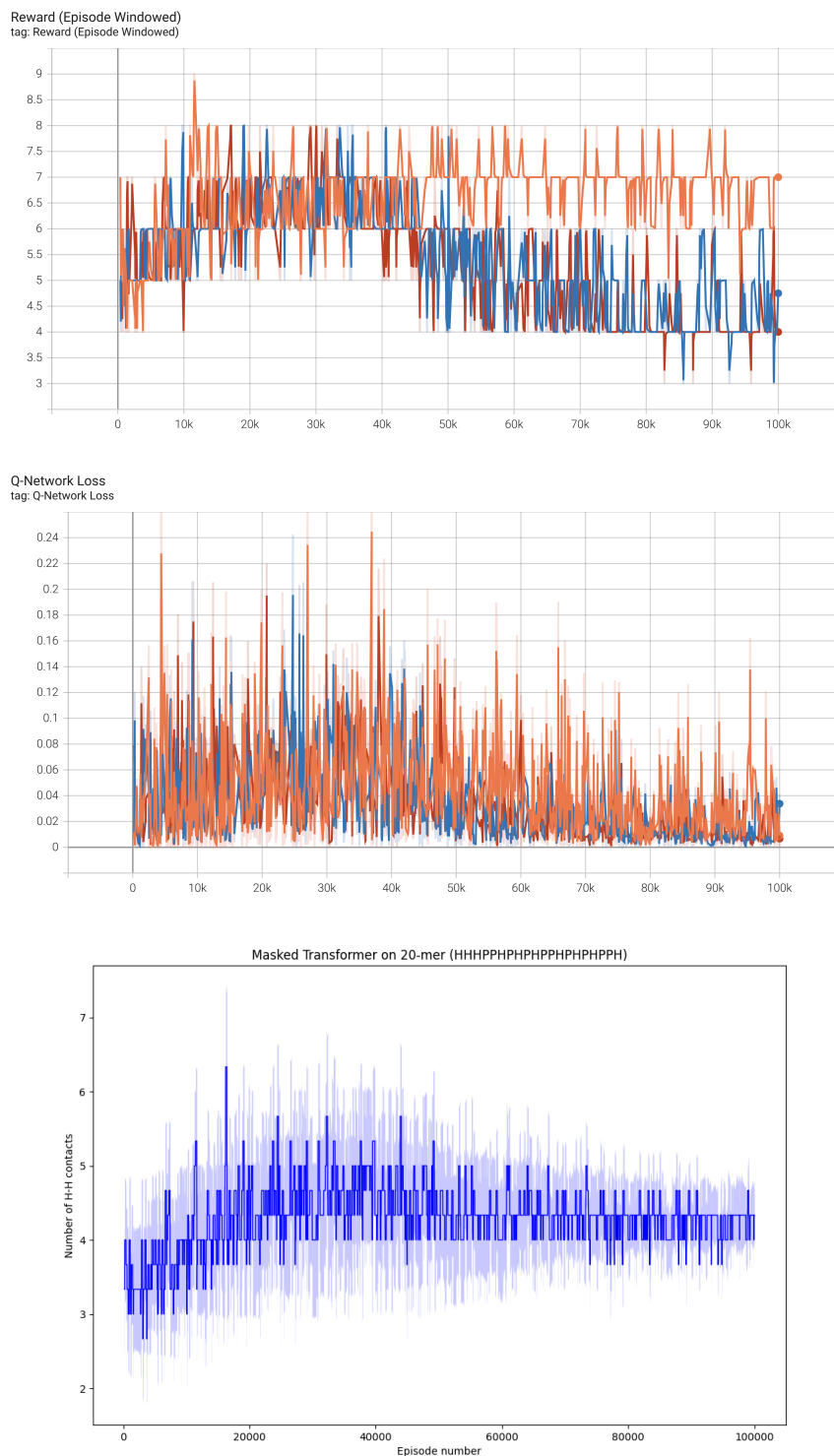


Figure 12: This figure shows the performance of the Masked Transformer model on the test 20-mer sequence (HHHPPHPHPHPHPHPHPH). *Top:* The maximum reward received over a rolling 200-episode window by each of the three seeds (1, 2, 3) tested. *Middle:* The Q-Network loss for each seed. *Bottom:* The maximum reward received over a rolling 200-episode window over the average of the three seeds. The shaded area represents 1 standard deviation above and below the mean. Yash

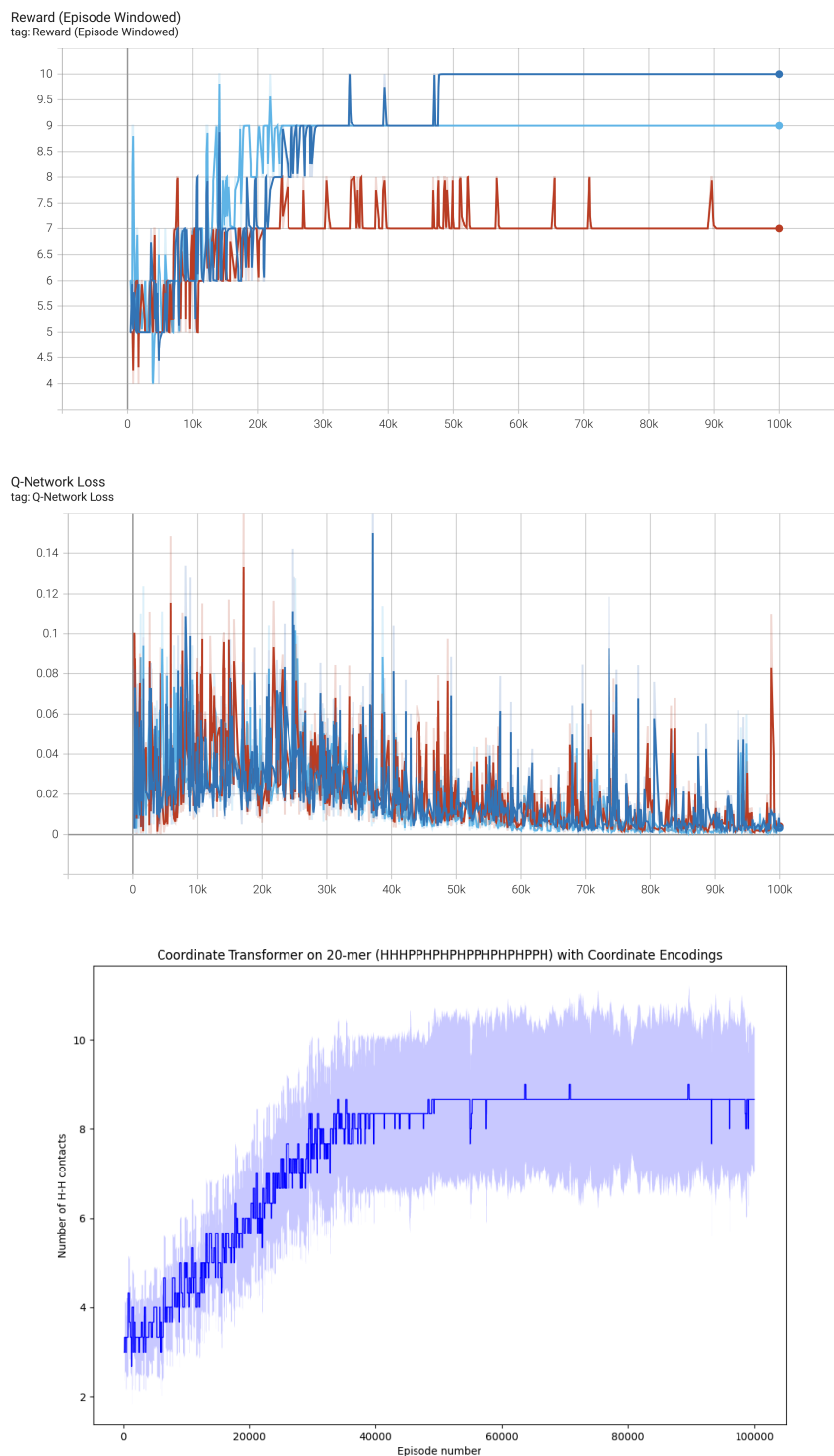


Figure 13: This figure shows the performance of the Coordinate Transformer model on the test 20-mer sequence (HHHPPHPHPHPHPHPHPH) with Coordinate Encodings (Section 4.8). *Top:* The maximum reward received over a rolling 200-episode window by each of the three seeds (1, 2, 3) tested. *Middle:* The Q-Network loss for each seed. *Bottom:* The maximum reward received over a rolling 200-episode window over the average of the three seeds. The shaded area represents 1 standard-deviation above and below the mean. **Yash**

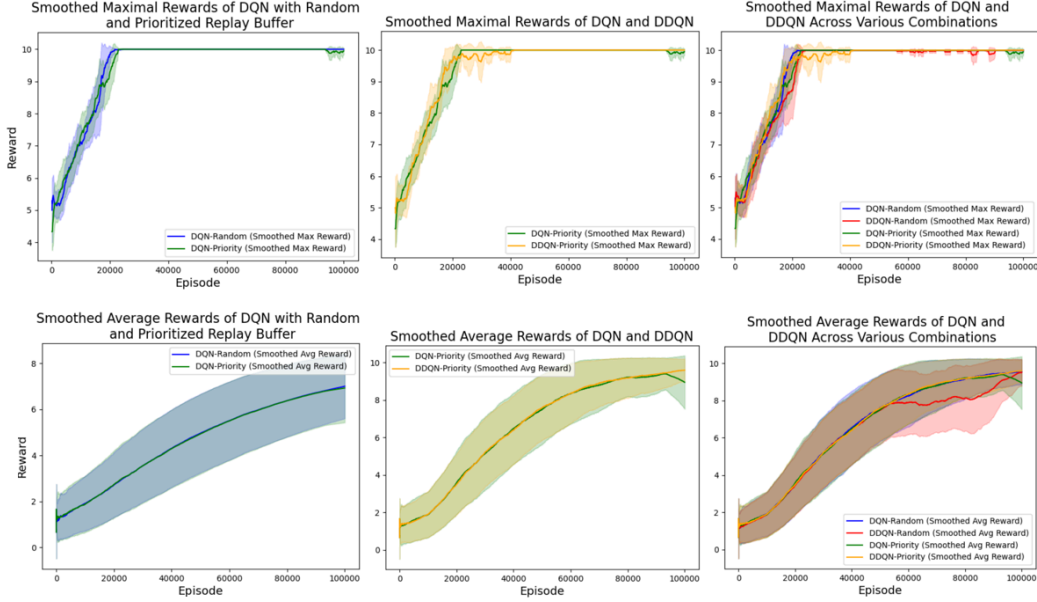


Figure 14: The upper panel displays the maximum reward over a 200-episode window, while the lower panel presents the average reward, smoothed over 1,000 episodes. The left plot compares the smoothed average reward for DQN using both random (blue) and prioritized (green) replay buffers. The middle plot compares the smoothed average reward for DQN (green) and DDQN (orange), both using PER. The right plot combines all variations, showing DQN and DDQN with both random and prioritized replay buffers. All graphs averaged and with standard deviation, calculated from three different seeds: 42, 0, and 123. The results confirm that all methods converge around the same episode of 20K. [Tali](#)

Our primary objective was to determine whether the DDQN agent improves convergence by mitigating the overestimation bias in DQN. However, as shown in Figure 14, the convergence behavior of DQN and DDQN is nearly identical, with no significant advantage observed for DDQN. Additionally, Figure 15 illustrates that the Q-values of both agents appear similar, with only a slight indication of overestimation in DQN compared to DDQN. [Tali](#)

Several factors could explain this result. First, as noted in (Lan et al. (2020)), DDQN does not always outperform DQN, its benefits vary across different environments. Second, hyperparameter choices can lead to similar behavior in both DQN and DDQN. For instance, if the exploration rate is too low, the agent is exposed to a limited set of state-action pairs, resulting in comparable learning dynamics. Third, according to (Wagenbach and Sabatelli (2022)), overestimation bias is most pronounced in highly stochastic environments. If our environment lacks sufficient stochasticity, the benefits of DDQN’s overestimation correction may be minimal. It is possible that DDQN would become more advantageous in longer sequences, where reward variability is higher. However, due to computational constraints, we were unable to extend this experiment to such sequences. [Tali](#)

5.8 PRIORITIZED REPLAY BUFFER VERSUS RANDOM APPROACH

We also experimented with a replay buffer that prioritizes transitions with higher TD error, aiming to improve sample efficiency and accelerate learning. To achieve this, we needed to implement a prioritization data structure to keep computational overhead minimal. To efficiently maintain prioritized sampling, we implemented a sum-tree and a min-tree data structure. These structures are similar to the array representation of a binary heap, reducing the complexity of updating values from $O(N)$ to $O(\log N)$. The sum-tree stores the sum of all priorities, enabling efficient probability normalization for sampling transitions. The min-tree tracks the minimum priority, which is crucial for normalizing importance-sampling weights. [Tali](#)

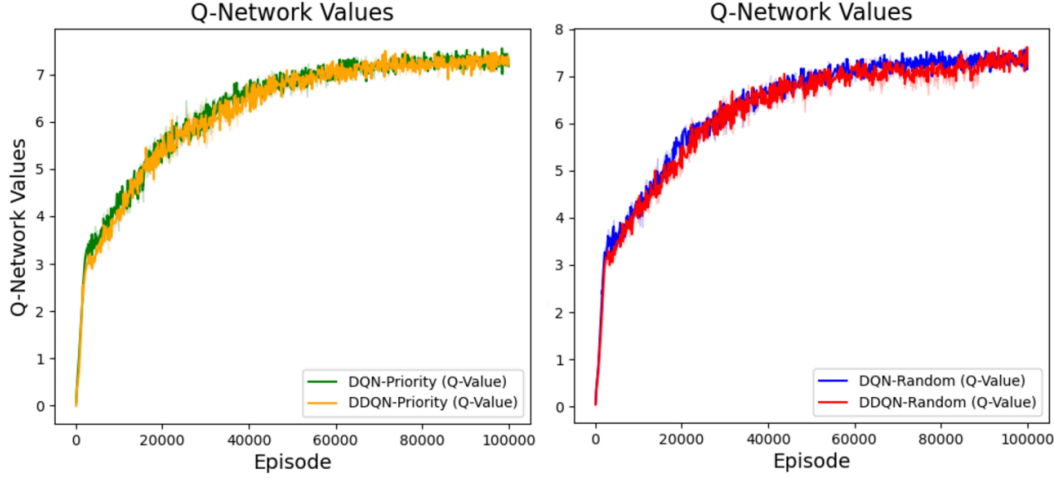


Figure 15: The left plot shows the Q-network values when using a prioritized PER for both DQN (green) and DDQN (orange). The right plot illustrates Q-network values using uniform experience replay for DQN (blue) and DDQN (red). The values averaged and with standard deviation, calculated from three different seeds: 42, 0, and 123. (Tali)

We use importance sampling to correct for the bias introduced by prioritized experience replay (PER). Unlike uniform replay, where transitions are sampled randomly, PER assigns higher sampling probabilities to high TD error experiences, skewing the training distribution towards these high priority samples. This imbalance can lead to biased value estimates, as the sampled data no longer accurately represents the true distribution of experiences in the environment. More details on our implementation can be found in (labml.ai (2025), Schaul et al. (2015b)). (Tali)

As shown in Figure 16, PER introduces greater fluctuations in the loss function during early training, particularly in the first phase before stabilizing around 20K episodes. This is expected, as prioritizing high TD error transitions leads to more drastic weight updates initially. Without PER, convergence time is approximately 30 minutes, whereas with PER, training takes around 43 minutes due to the computational overhead of maintaining priorities and sampling efficiently. Furthermore, we observe that DDQN consistently exhibits lower loss values than DQN, as DDQN reduces overestimation bias, leading to smaller TD errors and more stable updates. However, despite these differences in loss behavior, Figure 14 shows that the overall convergence patterns of DQN and DDQN, with or without PER, remain similar. This result suggests that PER does not provide a significant advantage in this environment, likely due to lack of rare critical transitions. If most sampled experiences are not particularly rare or important, uniform replay might already be sufficient for efficient learning. (Tali)

5.9 CHALLENGES IN FINE-TUNING FOR ENVIRONMENT GENERALIZATION

Seeing the fast convergence of PPO, we decided to use it for our next set of experiments. As previously mentioned, hyperparameter selection is highly sequence-dependent. To address this, we pretrained batches E_b of environment E using the optimal number of collected trajectories and epochs found for policy updates, shown in Table 1. The specific configurations for different sequence lengths were: $E_1 = \{4, 6, 20, 20\}$, $E_2 = \{24, 25\}$, $E_3 = \{36, 48, 50\}$, $E_4 = \{100\}$, where each component represents the length of a sequence. The number of training episodes for each sequence batch E_b was set to 30K, 50K, 150K, and 100K, respectively, using an LSTM with a hidden size of 64 and three layers. (Tali)

We examined different types of learning configurations, focusing on two primary approaches: random learning and curriculum learning. In random learning, the sequences introduced to the agent at each E_b training phase were randomly selected from the dataset. In

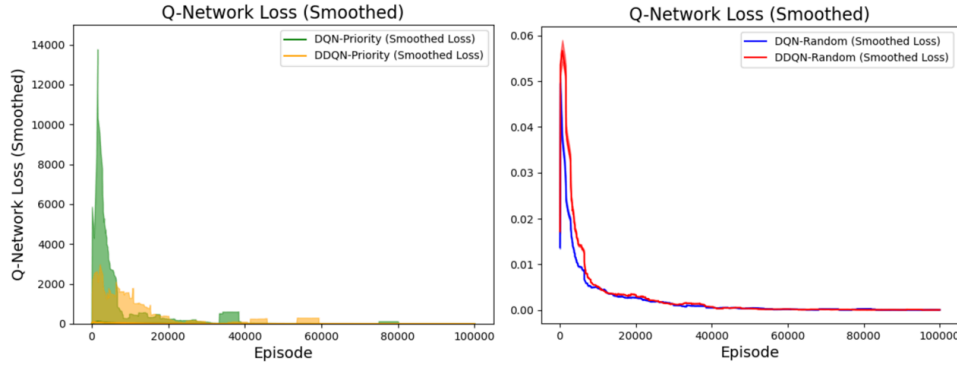


Figure 16: The left plot shows the Q-network loss when using a prioritized experience replay buffer (PER) for both DQN (green) and DDQN (orange), smoothed across 50 episodes window. The right plot illustrates Q-network loss using uniform experience replay for DQN (blue) and DDQN (red), smoothed across 50 episodes window. The values averaged and with standard deviation, calculated from three different seeds: 42, 0, and 123. (Tali)

contrast, curriculum learning presented sequences progressively, starting from simpler ones and gradually increasing in complexity. To determine when to transition to a more difficult sequence in curriculum learning, we tested different strategies. Ultimately, we set a threshold based on the normalized average reward (scaled by its highest optimal value). When the agent exceeded this threshold (0.5), a new sequence was introduced with a probability of 0.8, while still allowing the model to occasionally (with a probability of 0.2) revisit earlier sequences to maintain previously learned knowledge. (Tali)

As shown in Figure 17, random learning resulted in more stable value loss but at a higher level as well as stable and lower entropy. This stability can be attributed to the fact that the agent encounters different sequences randomly, leading to the development of more generalized value estimates. Also, the agent learns a generalized policy early on, maintaining a consistent level of exploration across all training episodes. Because the policy does not overfit to any particular sequence before transitioning to another, entropy remains steady, preventing premature convergence.

In contrast, curriculum learning exhibited fluctuations pararerally in value loss and entropy due to introducing new, increasingly difficult sequences. It exhibits an initial increase in entropy before stabilizing, followed by a sharp decline whenever a new sequence is intro-

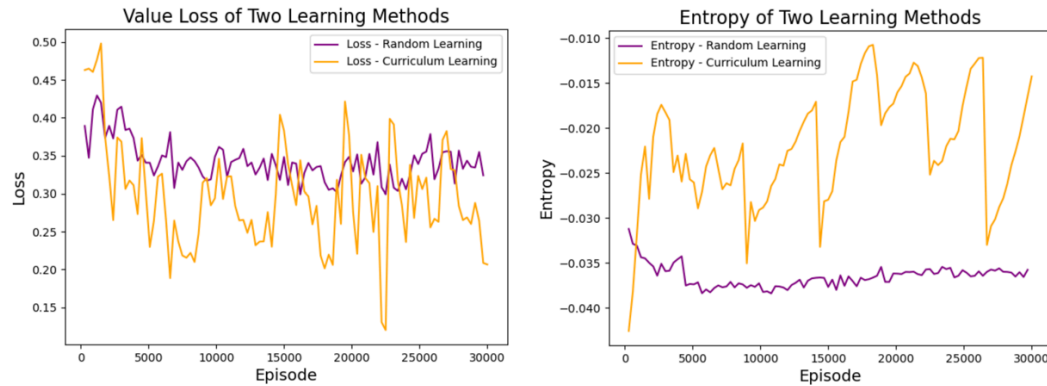


Figure 17: The left plot shows the value loss for random learning (purple) and curriculum learning (orange). The right plot illustrates entropy over training. The curriculum learning model here was set to focus exclusively on one sequence at a time, without occasional returns to previously learned sequences, resulting in stronger fluctuations in value loss and entropy. (Tali)

duced. One possible explanation for this behavior is that when a new sequence appears, the agent reduces exploration in an attempt to minimize risk in the unfamiliar scenario. As a result, the agent may restrict itself to fewer actions, making the policy more deterministic to avoid errors caused by uncertainty. However, as the agent adapts to the new sequence, exploration gradually increases again, allowing it to refine its policy. (Tali)

We noticed that introducing a new sequence sometimes led to a drop in rewards for previously learned sequences. This suggests that older policies were overwritten or destabilized when new knowledge was acquired. Furthermore, even in shorter sequences, we did not always achieve the highest possible reward, although curriculum learning performed slightly better than random learning despite its spikes in loss and entropy. Lastly, we tested whether pretraining on shorter sequences would accelerate learning on longer sequences, but this approach did not lead to higher rewards or faster convergence (the highest reward achieved on 100mer was 27, compared to 32 with no pretraining—Table 1). This further emphasizes the strong dependency of each sequence on its specific hyperparameters and highlights the challenges of tuning the PPO algorithm effectively across different tasks. (Tali)

6 FUTURE DIRECTIONS AND LIMITATIONS

- Building on our demonstration of the importance of capturing long-range dependencies, we propose the use of graph rewiring techniques derived from discrete geometry (Bober et al. (2023)) and probabilistic approaches (Qian et al. (2024)) for mitigating the under-reaching (Lu et al. (2024)) (and also potentially over-squashing (Topping et al. (2021))) issue encountered with the use of GNNs on this task. (Yash)
- Investigating and resolving the instability post-convergence encountered with the models, particularly Mamba, GNN-based methods, and PPO. (Yash)
 - ▶ A possible direction could be using a learning rate schedule that adapts to the current loss (Takase et al. (2018)). (Yash)
 - ▶ Normalization layers such as Batch (Ioffe and Szegedy (2015)) and Layer Normalization could be explored to avoid problems such as internal covariate shift that could be plaguing all models tested as well as using GNN-specific normalization strategies (Eliasof et al. (2024)). (Yash)
 - ▶ One possible direction could be the use of Polyak averaging (Li et al. (2023)) with DQN to update the target Q-function, instead of the abrupt copying of weights from the current policy. This could possibly lead to smoother updates that could help avoid training instability. (Yash)
 - ▶ For PPO, instead of using the traditional advantage method, we can use Generalized Advantage Estimation (GAE) (Schulman et al. (2015)), which combines multiple n-step TD estimates instead of a single step to improve variance reduction without significantly increasing bias. (Tali)
- Replacing the absolute Coordinate Encodings scheme with a relative Rotary Coordinate Encoding scheme (Su et al. (2024)) to preserve translation invariance, enhancing a model's ability to recognize patterns appearing at different positions in the HP sequence. (Yash)
- Since we found that PPO is a better agent than DQN and MAMBA performs as a better model than LSTM, a future direction will be experimenting with this combination (our code already supports this, but we were unable to perform runs due to time and computational constraints). (Tali)
- Future research could build upon the findings in Table 1 by conducting an investigation into hyperparameters tuning. This has the potential to establish a scaling law where the length of a sequence predicts the number of episodes required for convergence. Such insight could enhance predictive modelling and efficiency in the

HP model training process. Furthermore, effective research into hyperparameters might also facilitate the development of generalized models. Tali

7 CONCLUSION

- **PPO Performance:** PPO is generally faster at converging to the optimal value compared to DQN, but its effectiveness can diminish without specific tuning for each sequence. For sequences where $N < 36$, we achieved convergence rates 2–7 times faster than DQN baseline. This highlights the need for more research on hyperparameters tuning, particularly for longer sequences. Tali
- **Alternate model architectures:** We found Mamba to be an exceptionally effective model for this task, finding the optimal folding and converging much faster than the previous LSTM-based method. The transformer took longer to find the optimal foldings, which we suspect is due to the fact that it requires much more data to train. Yash
- **Importance of long-range dependency modelling:** We demonstrated with the use of attention masking on our transformer that models need to be able to propagate information effectively across long distances to perform well at this task. This also explains the failure of GAT, as it relies on local information propagation via message passing. Yash
- **DQN Variants:** We observed no significant differences in performance between DQN, DDQN, and DQN with PER. This suggests that these enhancements may not provide substantial benefits for our specific tasks, though they could potentially be more useful for longer sequences. Tali
- **Generalization:** Our attempts to utilize a pre-trained model for generalization across different sequence lengths were unsuccessful. This indicates a challenge in applying generalized models to longer sequences, emphasizing the complexity of hyperparameter tuning in the PPO algorithm. Furthermore, in this setting, curriculum learning slightly outperformed random training, achieving higher scores. Tali

REFERENCES

- Anfinsen, C. B. Principles that govern the folding of protein chains. *Science*, 181(4096), 223–230, 1973.
- Atwood, J. L. *Comprehensive supramolecular chemistry II*. Elsevier, 2017.
- Bahdanau, D., Cho, K., and Bengio, Y. Neural machine translation by jointly learning to align and translate. *Arxiv Preprint Arxiv:1409.0473*, 2014.
- Berger, B., and Leighton, T. Protein folding in the hydrophobic-hydrophilic (HP) is NP-complete. *Proceedings of the Second Annual International Conference on Computational Molecular Biology*, 30–39, 1998.
- Bianchi, F. M., and Lachi, V. The expressive power of pooling in graph neural networks. *Advances in Neural Information Processing Systems*, 36, 71603–71618, 2023.
- Bober, J., Monod, A., Saucan, E., and Webster, K. N. Rewiring networks for graph neural network training using discrete geometry. *International Conference on Complex Networks and Their Applications*, 225–236, 2023.
- Bronstein, M. M., Bruna, J., LeCun, Y., Szlam, A., and Vandergheynst, P. Geometric deep learning: going beyond euclidean data. *IEEE Signal Processing Magazine*, 34(4), 18–42, 2017.

- Chen, M., and Huang, W.-Q. A branch and bound algorithm for the protein folding problem in the HP lattice model. *Genomics, Proteomics & Bioinformatics*, 3(4), 225–230, 2005.
- Czibula, G., Bocicor, M.-I., and Czibula, I.-G. A reinforcement learning model for solving the folding problem. *International Journal of Computer Technology and Applications*, 2(1), 2011.
- Daggett, V. Protein folding- simulation. *Chemical Reviews*, 106(5), 1898–1916, 2006.
- Deng, M., Wang, J., Hsieh, C.-P., Wang, Y., Guo, H., Shu, T., Song, M., Xing, E. P., and Hu, Z. Rlprompt: Optimizing discrete text prompts with reinforcement learning. *Arxiv Preprint Arxiv:2205.12548*, 2022.
- Dill, K. A., Bromberg, S., Yue, K., Chan, H. S., Ftebig, K. M., Yee, D. P., and Thomas, P. D. Principles of protein folding—a perspective from simple exact models. *Protein Science*, 4(4), 561–602, 1995.
- Dinner, A. R., Šali, A., Smith, L. J., Dobson, C. M., and Karplus, M. Understanding protein folding via free-energy surfaces from theory and experiment. *Trends in Biochemical Sciences*, 25(7), 331–339, 2000.
- Dobson, C. M. Protein folding and misfolding. *Nature*, 426(6968), 884–890, 2003.
- Doğan, B., and Ölmez, T. A novel state space representation for the solution of 2D-HP protein folding problem using reinforcement learning methods. *Applied Soft Computing*, 26, 213–223, 2015.
- Eliasof, M., Bevilacqua, B., Schönlieb, C.-B., and Maron, H. Granola: Adaptive normalization for graph neural networks. *Arxiv Preprint Arxiv:2404.13344*, 2024.
- Engstrom, L., Ilyas, A., Santurkar, S., Tsipras, D., Janoos, F., Rudolph, L., and Madry, A. Implementation matters in deep rl: A case study on ppo and trpo. *International Conference on Learning Representations*, 2019.
- Espitia, G., Pang, Y. T., and Gumbart, J. C. Protein Structure Prediction in the 3D HP Model Using Deep Reinforcement Learning. *Arxiv Preprint Arxiv:2412.20329*, 2024.
- Gambardella, L. M., and Dorigo, M. Ant-Q: A reinforcement learning approach to the traveling salesman problem. In *Machine learning proceedings 1995*, pp. 252–260. Elsevier, 1995.
- Gu, A., and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *Arxiv Preprint Arxiv:2312.00752*, 2023.
- Gu, A., Dao, T., Ermon, S., Rudra, A., and Ré, C. Hippo: Recurrent memory with optimal polynomial projections. *Advances in Neural Information Processing Systems*, 33, 1474–1487, 2020.
- Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. *Arxiv Preprint Arxiv:2111.00396*, 2021.
- Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. Dream to control: Learning behaviors by latent imagination. *Arxiv Preprint Arxiv:1912.01603*, 2019.
- Honig, B., and Yang, A.-S. Free energy balance in protein folding. *Advances in Protein Chemistry*, 46, 27–58, 1995.
- Hoque, T., Chetty, M., and Sattar, A. Extended HP model for protein structure prediction. *Journal of Computational Biology*, 16(1), 85–103, 2009.
- Ioffe, S., and Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *International Conference on Machine Learning*, 448–456, 2015.

- Khandelwal, U., He, H., Qi, P., and Jurafsky, D. Sharp nearby, fuzzy far away: How neural language models use context. *Arxiv Preprint Arxiv:1805.04623*, 2018.
- labml.ai. *Prioritized Replay Buffer*, 2025. https://github.com/labmlai/annotated_deep_learning_paper_implementations/blob/master/labml_nn/rl/dqn/replay_buffer.py
- Lan, Q., Pan, Y., Fyshe, A., and White, M. Maxmin q-learning: Controlling the estimation bias of q-learning. *Arxiv Preprint Arxiv:2002.06487*, 2020.
- Li, X., Yang, W., Liang, J., Zhang, Z., and Jordan, M. I. A statistical analysis of polyak-ruppert averaged q-learning. *International Conference on Artificial Intelligence and Statistics*, 2207–2261, 2023.
- Li, Y., Kang, H., Ye, K., Yin, S., and Li, X. Foldingzero: Protein folding from scratch in hydrophobic-polar model. *Arxiv Preprint Arxiv:1812.00967*, 2018.
- Liu, Y., Luo, Y., Zhong, Y., Chen, X., Liu, Q., and Peng, J. Sequence modeling of temporal credit assignment for episodic reinforcement learning. *Arxiv Preprint Arxiv:1905.13420*, 2019.
- Louie, K. Asymmetric and adaptive reward coding via normalized reinforcement learning. *Plos Computational Biology*, 18(7), e1010350, 2022.
- Lu, W., Guan, Z., Zhao, W., Yang, Y., and Jin, L. Nodemixup: Tackling under-reaching for graph neural networks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(13), 14175–14183, 2024.
- Narvekar, S., Peng, B., Leonetti, M., Sinapov, J., Taylor, M. E., and Stone, P. Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, 21(181), 1–50, 2020.
- Qian, C., Manolache, A., Morris, C., and Niepert, M. Probabilistic graph rewiring via virtual nodes. *Arxiv Preprint Arxiv:2405.17311*, 2024.
- Reiser, P., Neubert, M., Eberhard, A., Torresi, L., Zhou, C., Shao, C., Metni, H., Hoesel, C. van, Schopmans, H., Sommer, T., and others. Graph neural networks for materials science and chemistry. *Communications Materials*, 3(1), 93, 2022.
- Ruoss, A., Delétang, G., Genewein, T., Grau-Moya, J., Csordás, R., Bennani, M., Legg, S., and Veness, J. Randomized positional encodings boost length generalization of transformers. *Arxiv Preprint Arxiv:2305.16843*, 2023.
- Schaul, T., Quan, J., Antonoglou, I., and Silver, D. Prioritized experience replay. *Arxiv Preprint Arxiv:1511.05952*, 2015a.
- Schaul, T., Quan, J., Antonoglou, I., and Silver, D. Prioritized experience replay. *Arxiv Preprint Arxiv:1511.05952*, 2015b.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. *Arxiv Preprint Arxiv:1506.02438*, 2015.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *Arxiv Preprint Arxiv:1707.06347*, 2017.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 127063, 2024.
- Sutton, R., and Barto, A. Reinforcement Learning: An Introduction. *IEEE Transactions on Neural Networks*, 9(5), 1054, 1998. <https://doi.org/10.1109/TNN.1998.712192>
- Takase, T., Oyama, S., and Kurihara, M. Effective neural network training with adaptive learning rate based on training loss. *Neural Networks*, 101, 68–78, 2018.

- Tay, Y., Dehghani, M., Abnar, S., Shen, Y., Bahri, D., Pham, P., Rao, J., Yang, L., Ruder, S., and Metzler, D. Long range arena: A benchmark for efficient transformers. *Arxiv Preprint Arxiv:2011.04006*, 2020.
- Topping, J., Di Giovanni, F., Chamberlain, B. P., Dong, X., and Bronstein, M. M. Understanding over-squashing and bottlenecks on graphs via curvature. *Arxiv Preprint Arxiv:2111.14522*, 2021.
- Van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1), 2016a.
- Van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1), 2016b.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- Veličković, P. Message passing all the way up. *Arxiv Preprint Arxiv:2202.11097*, 2022.
- Veličković, P. Everything is connected: Graph neural networks. *Current Opinion in Structural Biology*, 79, 102538, 2023.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., and Bengio, Y. Graph attention networks. *Arxiv Preprint Arxiv:1710.10903*, 2017.
- Wagenbach, J., and Sabatelli, M. Factors of Influence of the Overestimation Bias of Q-Learning. *Arxiv Preprint Arxiv:2210.05262*, 2022.
- Wu, H., Yang, R., Fu, Q., Chen, J., Lu, W., and Li, H. Research on predicting 2D-HP protein folding using reinforcement learning with full state space. *BMC Bioinformatics*, 20, 1–11, 2019.
- Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., and Philip, S. Y. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1), 4–24, 2020.
- Yang, K., Huang, H., Vandans, O., Murali, A., Tian, F., Yap, R. H., and Dai, L. Applying deep reinforcement learning to the HP model for protein structure prediction. *Physica A: Statistical Mechanics and Its Applications*, 609, 128395, 2023.
- Yu, H., Schreck, J. S., and Ma, W.-J. Deep reinforcement learning for protein folding in the hydrophobic-polar model with pull moves. *Neural Information Processing Systems (Neurips): Workshop on Machine Learning and the Physical Sciences*, 2020.
- Yu, Y., Si, X., Hu, C., and Zhang, J. A review of recurrent neural networks: LSTM cells and network architectures. *Neural Computation*, 31(7), 1235–1270, 2019.
- Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., and Sun, M. Graph neural networks: A review of methods and applications. *AI Open*, 1, 57–81, 2020.